

Verifiable Decapsulation: Recognizing Faulty Implementations of Post-Quantum KEMs

Lewis Glabush¹, Felix Günther², Kathrin Hövelmanns³, and Douglas Stebila⁴

¹ École Polytechnique Fédérale de Lausanne, Switzerland
`lewis.glabush@epfl.ch`

² IBM Research Europe – Zurich, Switzerland
`mail@felixguenther.info`

³ Eindhoven University of Technology, The Netherlands
`kathrin@hoevelmanns.net`

⁴ University of Waterloo, Canada
`dstebila@uwaterloo.ca`

Abstract. Cryptographic schemes often contain verification steps that are essential for security. Yet, faulty implementations missing these steps can easily go unnoticed, as the schemes might still function correctly. A prominent instance of such a verification step is the re-encryption check in the Fujisaki–Okamoto (FO) transform that plays a prominent role in the post-quantum key encapsulation mechanisms (KEMs) considered in NIST’s PQC standardization process. In KEMs built from FO, decapsulation performs a re-encryption check that is essential for security, but not for functionality. In other words, it will go unnoticed if this essential step is omitted or wrongly implemented, opening the door for key recovery attacks. Notably, such an implementation flaw was present in HQC’s reference implementation and was only noticed after 19 months.

In this work, we develop a modified FO transform that binds re-encryption to functionality, ensuring that a faulty implementation which skips re-encryption will be exposed through basic correctness tests. We do so by adapting the “verifiable verification” methodology of Fischlin and Günther (CCS 2023) to the context of FO-based KEMs. More concretely, by exporting an unpredictable confirmation code from the public key encryption and embedding it into the key derivation function, we can confirm that (most of) the re-encryption step was indeed performed during decapsulation. We formalize this concept, establish modified FO transforms, and prove how unpredictable PKE confirmation codes turn into noticeable correctness errors for faulty implementations. We show how to apply this technique to ML-KEM and HQC, both with negligible overhead, by leveraging the entropy lost through ciphertext compression or truncation. We confirm that our approach works through mathematical proofs, as well as experimental data. Our experiments show that the implementation flaw in HQC’s reference implementation indeed makes basic test cases fail when following our approach.

Keywords: Key encapsulation mechanism, public-key encryption, Fujisaki–Okamoto transformation, NIST, ML-KEM, HQC, post-quantum security, QROM

We thank Bénédict Tran, Bertram Poettering, Serge Vaudenay, Spencer Wilson, Vadim Lyubashevsky, and Britta Hale for insightful discussions, and the anonymous reviewers for their valuable feedback. K.H. was supported by an NWO VENI grant (Project No. VI.Veni.222.397). D.S. was supported by Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery grant RGPIN-2022-03187 and NSERC Alliance grant ALLRP 578463-22.

1 Introduction

A key encapsulation mechanism (KEM) allows two parties to establish a shared secret key using only public communication. Post-quantum KEMs, like NIST’s recent standard ML-KEM [23], are at the heart of the current transition to quantum-safe cryptography. The most widespread approach used in constructing post-quantum KEMs is to design a public-key encryption (PKE) scheme and then apply the Fujisaki–Okamoto (FO) transform [10, 11] that turns any weakly secure PKE scheme into an IND-CCA-secure KEM using de-randomization and a re-encryption check.

THE FO TRANSFORM. In a nutshell, the FO transform builds KEM encapsulation by sampling a random message m , PKE-encrypting m into a ciphertext c with randomness r derived from m , and outputting c and a shared secret derived from m . Decapsulation PKE-decrypts c into m' , derives randomness r' from m' , *re-encrypts* m' using r' into c' , and *checks* that $c = c'$ before accepting with the key derived from m' . This is what is called the FO re-encryption check.

BRITTLE CRYPTOGRAPHY. When implementing cryptographic schemes and protocols, every detail matters and even small implementation bugs can have devastating effects on security. In that regard, cryptographic code can be particularly brittle, especially when some cryptographic verification steps are accidentally skipped or wrongly executed: from a functional point of view, such bugs can be entirely unnoticeable, yet render an implementation completely insecure. A prime example of such a bug was the “goto fail” bug [26] in Apple’s SSL/TLS library, where a misplaced `goto fail`; code line resulted in crucial certificate validation steps being skipped, causing invalid certificates to pass validation.

The FO re-encryption check is prone to similar implementation flaws: skipping or wrongly implementing this check could go unnoticed (as honestly generated ciphertexts decrypt correctly), yet may open the door even for key recovery attacks (when the IND-CCA security guarantee of the FO transform becomes voided). Indeed, such an implementation flaw was present until recently in the reference implementation of HQC [2], a Round 4 submission in NIST’s PQC standardization process selected for standardization by now, and was picked up by downstream users of that code, such as `liboqs` [24, 29]. The decapsulation code would accept malformed ciphertexts due to an unfortunate interaction of flawed calculations during re-encryption and a logical flaw checking its result.⁵ Despite common test cases for correctness (“Does the shared secret output by encapsulation equal the shared secret output by decapsulation?”), this flaw was only noticed after 19 months [25].

MAKING DECAPSULATION RECOGNIZE IMPLEMENTATION FAULTS. In this work, we ask how to better protect implementations of FO-transformed KEMs from accidentally skipping or wrongly implementing the re-encryption check. Put differently, we want to devise a cryptographic approach to surface such implementation flaws in a noticeable manner, offering some assurance that a correct implementation does not miss certain security-crucial steps.

Our approach follows a methodology recently introduced by Fischlin and Günther [9], which in turn is inspired by an idea of Heninger [12], namely to minimize the impact of inevitable human errors in implementations by “tying security to basic functionality.” In essence, a security-critical implementation flaw should ideally noticeably affect the scheme’s functionality, meaning that it will be noticed early on in basic correctness or interoperability tests.

⁵ During the re-encryption part of HQC’s decapsulation code, the pseudorandom seed for derandomization did not include all the inputs used in the corresponding derandomized encapsulation, which lead to the wrong ciphertext being reconstructed. However, the complex constant-time code to check equality of the reconstructed and original ciphertexts *also* had a flaw that effectively flipped the logical result, meaning the ciphertext would be accepted as valid, and the correct shared secret (based on the decapsulated secret and the original ciphertext) would be output.

Fischlin and Günther [9] introduced the notion of *confirmation codes*, output by a cryptographic scheme (in addition to its regular outputs). Confirmation codes are to be designed in a way that they are “unpredictable”: a faulty-but-benign implementation of critical verification steps should be unlikely to compute them correctly. Such unpredictable confirmation codes can then be used in an overall protocol to ensure that the protocol will not function correctly (e.g., sender and receiver will be unable to establish a connection) if one side uses a faulty implementation. Note that unpredictability need not meet cryptographically small bounds, such as predictability probability less than 2^{-128} : unpredictability of even 2^{-1} means that a faulty-but-benign implementation will fail interoperability or correctness tests half the time, already flagging concerns about the implementation.

We transfer the idea of confirmation codes to the context of FO-based KEMs. By exporting an unpredictable confirmation code from the underlying PKE scheme and embedding it into the KEM’s key derivation function, we can confirm that the re-encryption step was indeed performed during decapsulation. Formally, we augment a PKE encryption algorithm with a confirmation code output and establish modified FO transforms (following the modularization by Hofheinz, Hövelmanns, and Kiltz [14]) which integrate this code into the KEM key derivation. Figure 1 illustrates the fault-recognizing KEM resulting from our confirmation-code-augmented FO transforms. We prove that unpredictable PKE confirmation codes turn into noticeable correctness errors on the KEM level when re-encryption is faultily implemented. We finally show how to apply this technique to ML-KEM and HQC with negligible overhead by leveraging the entropy lost due to rounding or truncating low-order ciphertext bits; the recipient can only recover this entropy if they indeed implement (nearly all of) re-encryption. We experimentally confirm that our approach works: it surfaces the flaw in HQC’s reference implementation by making basic test cases fail.

1.1 Contributions and Technical Overview

DEFINITIONS AND SECURITY NOTIONS FOR PKEs AND KEMs. We begin by defining the syntax and a security notion for confirmation-code-augmented public key encryption schemes, called *confirmation code unpredictability* (cUP). Compared to [9], we suggest a technically refined approach to capture the idea of a flawed implementation in which not all steps are executed. We model this through giving the adversary access to all inputs to the algorithm (which would of course allow them to trivially recompute everything the algorithm computes) but restricting the adversary’s access to some function F used by the algorithm. Looking ahead to our modifications to the FO transform, this limited access translates to the adversary being unable to fully/correctly implement the re-encryption step which depends on F . We define a related notion for KEMs, called *faulty implementation correctness* (fCOR), which models the ability of an adversarial faulty implementation to produce correct shared secrets during decapsulation, again with limited access to some function F used by the algorithm.

Let us emphasize that the core idea here is to catch *faulty-but-benign* implementations. We are not interested in capturing *malicious* implementations, which may of course skip re-encryption but make targeted calls to the restricted function F . The goal of our work, and the general idea of tying security to basic functionality [12] and making cryptographic operations verifiable [9], is to make accidental implementation errors be noticed to assist developers.

MODULAR FO TRANSFORMS FOR CONFIRMATION-CODE-AUGMENTED PKEs. We then present a sequence of modular transformations that build a fault-recognizing KEM from a PKE scheme with unpredictable confirmation codes such that the KEM has noticeable correctness errors if the FO re-encryption check is faultily implemented. Following [14] modularizing FO into transforms T and U , we give confirmation-code-augmented versions of these transforms TC and UC and the resulting confirmation-code-augmented FO transform FOC , for both the explicit and implicit rejection case.

Encaps(pk)	Decaps(sk, c)
01 $m \leftarrow_s \mathcal{M}$	05 $m' \leftarrow \text{Dec}(\text{sk}, c)$
02 $(c, \text{cd}) \leftarrow \text{Enc}(\text{pk}, m)$	06 $(c', \text{cd}') \leftarrow \text{Enc}(\text{pk}, m')$
03 $K \leftarrow \text{KDF}(m, \text{pk}, \text{cd})$	07 check $c' = c$
04 return (K, c)	08 $K' \leftarrow \text{KDF}(m', \text{pk}, \text{cd}')$
	09 return K'

Fig. 1: Simplified encapsulation and decapsulation algorithms of a fault-recognizing KEM, using our confirmation-code-augmented FO transform on a PKE scheme where encryption (Enc) additionally outputs a confirmation code cd . We highlight differences to a regular FO transform in violet boxes.

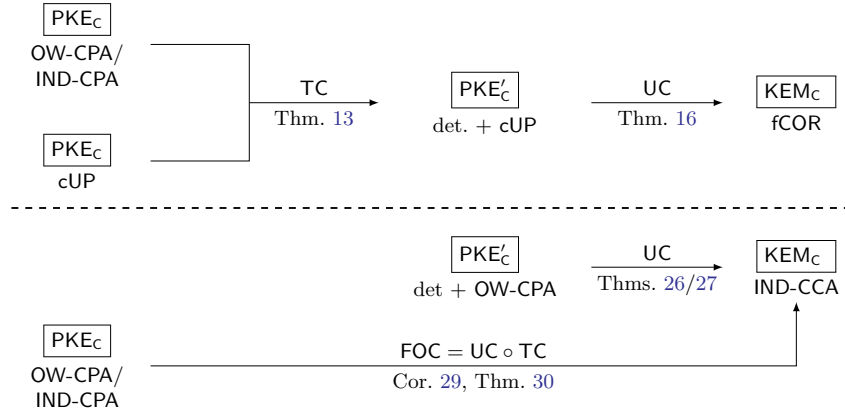


Fig. 2: Summary of our transformations to build a fault-recognizing KEM KEM_C from a confirmation-code-augmented PKE PKE_C using our $\text{FOC} = \text{UC} \circ \text{TC}$ transform. **Top:** Sequence of transformations/theorems to achieve faulty implementation correctness (fCOR) for KEM_C from confirmation-code unpredictability (cUP) and security of PKE_C . **Bottom:** Sequence of transformations/theorems to achieve IND-CCA-security for KEM_C from CPA security of PKE_C .

We show that TC maintains unpredictability and that FOC , like FO , turns a weakly (OW-CPA/IND-CPA) secure PKE scheme into an IND-CCA KEM; we provide these results in both the random oracle and quantum random oracle models. Most importantly, we prove that when using our FOC transforms on a PKE scheme with unpredictable confirmation codes (cUP), the resulting fault-recognizing KEM (illustrated in Fig. 1) will have noticeable correctness errors when re-encryption is faultily implemented (fCOR). Figure 2 summarizes these transformations and results.

APPLICATION TO ML-KEM AND HQC. We present a minimally modified version of ML-KEM [23] that augments the underlying PKE with confirmation codes based on the uncompressed module-LWE ciphertexts and includes them in the key derivation, following our $\text{FOC} = \text{UC} \circ \text{TC}$ transform. This introduces negligible overhead: using short confirmation codes of only 12–20 bytes ensures that a single correctness test fails with probability at least $\sim 1/3$, which will be highly noticeable in practice, and incurs at most 3.4% overhead in our experimental implementation.

We also present a variant of HQC [2] with confirmation codes. Here we use the last byte of a ciphertext component before truncation as the confirmation code; experimentally this incurs at most 0.25% overhead on runtime. We confirmed that adding a confirmation code to HQC would

have caught the bug HQC’s reference implementation [25]: with the confirmation code added, basic correctness tests (comparing whether the shared secret output by encapsulation equals the shared secret output by decapsulation) failed immediately in our experiments.

1.2 Related Work

The idea of tying security to basic functionality as a means to hedge against implementation errors was presented by Heninger [12] in a talk at the Workshop on Attacks in Cryptography 2 (WAC2) affiliated with Crypto 2019. Heninger discussed various flaws in deployed cryptographic implementations, suggesting that the “fragility under human error should be a cryptographic design consideration.” Inspired by this idea, Fischlin and Günther [9] introduced a methodology for “verifiable verification”, supporting the detection of implementation flaws on a cryptographic level. By letting verification algorithms output a confirmation code that can be used in higher-level protocols, they showed how to make faulty verification steps surface on the protocol level as noticeable correctness errors. The focus of [9] is on signature and MAC verification as well as elliptic curve point validation, and their usage in key exchange protocols. Our work applies the methodology to post-quantum KEMs and the FO transform, where the re-encryption step is similarly prone to implementation errors or accidental omission (possibly even intentional “optimization”), as also suggested by Heninger in a later talk at the Real World Post Quantum Workshop 2024 [13] and evidenced by the implementation flaw in HQC’s reference implementation [25].

During the NIST post-quantum standardization process, there has been a lot of work on the security analysis of FO-like transformations, with immense progress made stemming from the development of new QROM techniques. FO comes in two flavours with respect to the rejection mode used to reject invalid ciphertexts. (More details on this in Remark 6 in Section 2.2.) The FO variants with implicit rejection were proven secure against quantum attackers much earlier [14], with follow-up proofs allowing for successively better parameters [6, 17, 19, 21, 28], using successively more refined QROM techniques. For explicit rejection, there now also exist results if the base encryption scheme is probabilistic. Whereas the first proofs for explicit rejection [8, 20] did not allow for the same parameters as the ones for implicit rejection, two more recent results [16, 18] closed this gap by making use of an emerging new quantum proof technique, the extractable quantum random oracle [8]. If the base PKE scheme is deterministic, currently no security proofs for explicitly rejecting variants are known.

2 Preliminaries

In this section we recall the necessary background from previous literature that we will use throughout the paper: we recall definitions and notions for PKE schemes and KEMs, the Fujisaki–Okamoto transformation, and a helper result for the quantum-accessible ROM.

2.1 Public-Key Encryption

A public-key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ consists of three algorithms and a finite message space $\mathcal{M}$. The key generation algorithm KeyGen outputs a key pair (pk, sk) , with pk defining a randomness space $\mathcal{R} = \mathcal{R}(\text{pk})$. The encryption algorithm Enc , on input pk and a message $m \in \mathcal{M}$, produces an encryption $c \leftarrow \text{Enc}(\text{pk}, m)$ of m under the public key pk . If necessary, we explicitly specify the used randomness of encryption by writing $c = \text{Enc}(\text{pk}, m; r)$, where $r \leftarrow^* \mathcal{R}$. The decryption algorithm Dec , on input sk and a ciphertext c , yields either a message $m = \text{Dec}(\text{sk}, c) \in \mathcal{M}$ or a special symbol $\perp \notin \mathcal{M}$ to show that c is not a valid ciphertext.

Game FFP-ATK	ODec(c)
01 $(pk, sk) \leftarrow \text{KeyGen}$	06 $m \leftarrow \text{Dec}(sk, c)$
02 $m \leftarrow \mathcal{A}^{O_{\text{ATK}}, G'}(pk)$	07 return m
03 $c \leftarrow \text{Enc}(pk, m)$	
04 $m' \leftarrow \text{Dec}(sk, c)$	
05 return $\llbracket m' \neq m \rrbracket$	

Fig. 3: Games FFP-ATK for a deterministic PKE, where $\text{ATK} \in \{\text{CPA}, \text{CCA}\}$. O_{ATK} is the decryption oracle present in the respective game (see Definition 2) and G' is a random oracle, provided if it is used in the definition of PKE.

In the security analysis of FO-based KEMs, it is usually necessary to utilize notions of *correctness*. This is used to analyze the likelihood of a particular class of chosen-ciphertext attacks, where attackers found a failing ciphertext c , meaning c decrypts to a different message than its originating one, and uses this to obtain some leakage on the secret key. We now recall the information-theoretic correctness notion that was given in [14].

Definition 1 (PKE correctness [14]). *We call a public-key encryption scheme PKE δ -correct if*

$$\mathbf{E} \left[\max_{m \in \mathcal{M}} \Pr[\text{Dec}(sk, c) \neq m | c \leftarrow \text{Enc}(pk, m)] \right] \leq \delta,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}$.

This definition reflects that even a (possibly unbounded) adversary with access to the key pair cannot find failing messages with a probability higher than δ . A computational (game-based) approach that does not hand over the secret key was introduced in [16], via the *Find Failing Plaintext* (FFP) notions below.

Definition 2 (FFP-ATK). *Let $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a deterministic public-key encryption scheme. For $\text{ATK} \in \{\text{CPA}, \text{CCA}\}$, we define FFP-ATK games as in Fig. 3, where*

$$O_{\text{ATK}} = \begin{cases} - & \text{if } \text{ATK} = \text{CPA} \\ \text{ODec} & \text{if } \text{ATK} = \text{CCA} \end{cases}.$$

We define the FFP-ATK advantage function of an adversary $\mathcal{A}$ against PKE as

$$\text{Adv}_{\text{PKE}}^{\text{FFP-ATK}}(\mathcal{A}) = \Pr[\text{FFP-ATK}_{\text{PKE}}^{\mathcal{A}} \Rightarrow 1].$$

Definition 3 (Injectivity of PKE schemes [6]). *A deterministic PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ is ϵ -injective if*

$$\Pr[\text{Enc}(pk, m) \text{ is not injective} : (pk, sk) \leftarrow \text{KeyGen}()] \leq \epsilon.$$

We say PKE is injective if $\epsilon = 0$.

In the security analysis of FO-KEMs, it will also be necessary to capture that adversaries might be able to create valid ciphertexts without knowing the respective plaintext. This will involve the following notion about the entropy (or *spreadness*) of the PKE scheme.

Definition 4 (γ -spreadness). We say that PKE is γ -spread iff for all key pairs $(pk, sk) \in \text{supp}(\text{KeyGen})$ and all messages $m \in \mathcal{M}$ it holds that

$$\max_{c \in \mathcal{C}} \Pr[\text{Enc}(pk, m) = c] \leq 2^{-\gamma},$$

where the probability is taken over the internal randomness Enc .

We now recall the formal definitions of *One-Wayness* (OW-CPA) and *Indistinguishability under Chosen Plaintext Attacks* (CPA) security for PKE schemes.

Definition 5 (OW, IND-CPA). Given a PKE $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, we define the OW-CPA game as in Fig. 4, and the OW-CPA advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A}) = \Pr[\text{OW-CPA}_{\text{PKE}}(\mathcal{A}) \rightarrow 1] ,$$

where the probability is taken over the randomness in the OW-CPA game and the internal coins of $\mathcal{A}$.

We also define the IND-CPA game as in Fig. 4, and the IND-CPA advantage of an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ as

$$\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[\text{IND-CPA}_{\text{PKE}}(\mathcal{A}) \rightarrow 1] - \frac{1}{2} \right| ,$$

where the probability is taken over the randomness in the IND-CPA game and the internal coins of $\mathcal{A}$.

Game $\text{OW-CPA}_{\text{PKE}}(\mathcal{A})$:	Game $\text{IND-CPA}_{\text{PKE}}(\mathcal{A})$:
01 $(pk, sk) \leftarrow \text{KeyGen}$	06 $(pk, sk) \leftarrow \text{KeyGen}$
02 $m^* \leftarrow \mathcal{M}$	07 $b \leftarrow \mathcal{S}\{0, 1\}$
03 $c^* \leftarrow \text{Enc}(pk, m^*)$	08 $(m_0, m_1, st) \leftarrow A_1(pk)$
04 $m' \leftarrow \mathcal{A}(pk, c^*)$	09 $c^* \leftarrow \text{Enc}(pk, m_b)$
05 return $\llbracket m' = m \rrbracket$	10 $b' \leftarrow \mathcal{A}(pk, c^*, st)$
	11 return $\llbracket b' = b \rrbracket$

Fig. 4: OW-CPA and CPA game for PKE scheme PKE.

2.2 Fujisaki–Okamoto (FO) Transformation

In this section, we recall the definition of the FO transform as the composition of the two following transformations:

- the derandomizing **T-transform** that additionally adds a re-encryption check to the decryption procedure; and
- the PKE-to-KEM $\mathbf{U}_m$ -transforms that derive session keys from a randomly chosen message m , which they encrypt using PKE. The two variants of $\mathbf{U}_m$ vary in their responses to invalid ciphertexts ($\mathbf{U}_m^\perp$ returns $\perp$, while $\mathbf{U}_m^\mathcal{X}$ returns pseudo-random values).

The T-transform: To a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and a hash function $G' : \mathcal{M} \rightarrow \mathcal{R}$, we associate PKE scheme

$$\text{PKE}' = \mathbf{T}[\text{PKE}, G'] = (\text{KeyGen}', \text{Enc}', \text{Dec}') ,$$

KeyGen':	Enc'(pk, m):	Dec'(sk' = (pk, sk), c):
01 (pk, sk) $\leftarrow$ KeyGen	04 $c \leftarrow \text{Enc}(\text{pk}, m; G'(m))$	06 $m' \leftarrow \text{Dec}(\text{sk}, c)$
02 $\text{sk}' \leftarrow (\text{sk}, \text{pk})$	05 return c	07 if $m' = \perp$ or $c \neq \text{Enc}'(\text{pk}, m')$
03 return (pk, sk')		08 return $\perp$
		09 else
		10 return m'

Fig. 5: Algorithms of $T[\text{PKE}, G']$.

KeyGen ^x	Encaps _m (pk)
01 (pk, sk) $\leftarrow$ KeyGen	10 $m \leftarrow \mathcal{M}$
02 $s \leftarrow \mathcal{M}$	11 $c \leftarrow \text{Enc}(\text{pk}, m)$
03 $\text{sk}' \leftarrow (\text{sk}, s)$	12 $K \leftarrow G(m)$
04 return (pk, sk')	13 return (K, c)
Decaps _m [⊥] (sk, c)	Decaps _m ^x (sk', c)
05 $m' \leftarrow \text{Dec}(\text{sk}, c)$	14 $\text{Parse}(\text{sk}, s) \leftarrow \text{sk}'$
06 if $m' = \perp$	15 $m' \leftarrow \text{Dec}(\text{sk}, c)$
07 return $\perp$	16 if $m' = \perp$
08 else	17 return $K \leftarrow G(s, c)$
09 return $K \leftarrow G(m')$	18 else
	19 return $K \leftarrow G(m')$

Fig. 6: ‘Explicit rejection’ KEM $\text{KEM}_m^\perp = (\text{KeyGen}, \text{Encaps}_m, \text{Decaps}_m^\perp)$, and ‘implicit rejection’ KEM $\text{KEM}_m^x = (\text{KeyGen}^x, \text{Encaps}_m, \text{Decaps}_m^x)$, obtained from PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$.

with the algorithms defined in Fig. 5.

The U_m -transforms: To a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and a hash function $G : \mathcal{M} \rightarrow \mathcal{K}$, we associate key encapsulation mechanism

$$\begin{aligned} \text{KEM}_m^\perp &= U_m^\perp[\text{PKE}, G] = (\text{KeyGen}, \text{Encaps}_m, \text{Decaps}_m^\perp), \\ \text{KEM}_m^x &= U_m^x[\text{PKE}, G] = (\text{KeyGen}^x, \text{Encaps}_m, \text{Decaps}_m^x) \end{aligned}$$

where all algorithms are defined in Fig. 6.

Remark 6 (Rejection mode). In the literature, $\text{KEM}_m^\perp$ is often called ‘KEM with explicit rejection’ because decapsulation returns the dedicated failure symbol $\perp$ upon decryption failure. In turn, KEM_m^x is often called ‘KEM with implicit rejection’. This variant differs only from $\text{KEM}_m^\perp$ in that it reacts to invalid ciphertexts by returning a pseudo-random value instead of the dedicated failure symbol $\perp$ (compare line 17 to line 07).

We also briefly recall previous security results about these transformations: assuming a deterministic scheme PKE and modeling G as a random oracle, a tight relation has been proven between IND-CCA security of

- $\text{KEM}_m^\perp = U_m^\perp[\text{PKE}, G]$ and OW-VCA security of PKE [14, Theorem 3.5];
- $\text{KEM}_m^x = U_m^x[\text{PKE}, G]$ and OW-CPA security of PKE [14, Theorem 3.6].

2.3 Security Notions for Key Encapsulation Mechanisms

Here, we recall the standard security notions for key encapsulation: Indistinguishability under Chosen Ciphertext Attacks (IND-CCA).

Definition 7 (IND-CCA). We define the IND-CCA game as in Fig. 7 and the IND-CCA advantage function of an adversary $\mathcal{A}$ (with binary output) against KEM as

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr \left[\text{IND-CCA}_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1 \right] - \frac{1}{2} \right|.$$

Game $\text{IND-CCA}_{\text{KEM}}$:	$\text{Decaps}(c \neq c^*)$:
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}$	07 $K \leftarrow \text{Decaps}(\text{sk}, c)$
02 $b \leftarrow_{\$} \{0, 1\}$	08 return K
03 $(K_0, c^*) \leftarrow \text{Encaps}(\text{pk})$	
04 $K_1 \leftarrow_{\$} \mathcal{K}$	
05 $b' \leftarrow \mathcal{A}^{\text{Decaps}}(\text{pk}, c^*, K_b)$	
06 return $\llbracket b' = b^* \rrbracket$	

Fig. 7: IND-CCA game for $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$.

2.4 QROM helpers

We first recall one of the recent OWtH lemmata, Double-Sided OWtH [6, Lemma 5]. This lemma is used in the security analysis of FO-like KEMs to argue that a hash value (the output session key) is indistinguishable from random.

Lemma 8 (Double-Sided OWtH). Let $G, H : X \rightarrow Y$ be random functions, let z be a random value, and let $S \subset X$ be a random set such that $\forall x \in X \setminus S, G(x) = H(x)$. (G, H, S, z) may have arbitrary joint distribution. Let $\mathcal{A}^H$ be a quantum oracle algorithm. Let $f : X \rightarrow W \subset \{0, 1\}^n$ be any function, and let $f(S)$ denote the image of S under f . Let Ev be an arbitrary classical event. We will define another quantum oracle algorithm $\mathcal{B}^{G,H}(z)$. This $\mathcal{B}$ runs in about the same amount of time as $\mathcal{A}$, but when $\mathcal{A}$ queries H , $\mathcal{B}$ queries both G and H , and also runs f twice. Let

$$P_{\text{left}} = \Pr[\text{Ev} : \mathcal{A}^H(z)], P_{\text{right}} = \Pr[\text{Ev} : \mathcal{A}^G(z)], P_{\text{extract}} = \Pr[\mathcal{B}^{G,H}(z) \in f(S)].$$

If $f(S) = \{w^*\}$ is a single element, then $\mathcal{B}$ will only return $\perp$ or w^* , and furthermore,

$$|P_{\text{left}} - P_{\text{right}}| \leq 2\sqrt{P_{\text{extract}}}.$$

3 Confirmation-code-augmented PKE Schemes

In Section 4, we will augment variants of FO to tie functionality to security, using confirmation codes. This section provides the necessary formalism: in Section 3.1, we introduce the syntax and security notions for PKE schemes with added confirmation codes. To ease the analysis of probabilistic PKE schemes, in Section 3.2 we provide a confirmation-code-augmented counterpart of the derandomizing T-transform (which is part of the overall FO transform) and prove the necessary security results.

3.1 Confirmation-code-augmented PKE

We start by introducing the syntax of confirmation-code-augmented PKE (which we formalize as PKE_C), then adapt relevant standard definitions to the augmented setting, and finally formalize the main idea of confirmation codes as confirmation code unpredictability (cUP).

Definition 9 (Confirmation-code-augmented PKE). *A confirmation-code-augmented PKE scheme is a triplet of algorithms $\text{PKE}_C = (\text{KeyGen}, \text{Enc}_C, \text{Dec})$, together with message space $\mathcal{M}$, encryption randomness space $\mathcal{R}$ and confirmation code space $\mathcal{CD}$, such that*

- Enc_C is an algorithm that takes as input a public key pk , a message $m \in \mathcal{M}$ and encryption randomness $r \in \mathcal{R}$, and outputs a ciphertext c , together with a confirmation code $\text{cd} \in \mathcal{CD}$.
- $(\text{KeyGen}, \text{Enc}, \text{Dec})$ is a PKE scheme, where Enc denotes the algorithm which runs Enc_C and outputs only c .

We will later leverage that randomized PKE schemes involve some function F that we will model as a random oracle. We will make this oracle explicit in games. In such cases, we write KeyGen^F , Enc_C^F , and Enc^F to indicate the algorithms have (oracle) access to function F . (Deterministic algorithm Dec will not use F .)

While the definition might suggest that adding code confirmation introduces new computational steps and hence require additional resources, we will be able to instantiate confirmation codes by using intermediate values that were picked along the way while encrypting (see Section 5).

Definition 10 (Correctness and basic security for code-augmented PKE). *We say that a confirmation-code-augmented PKE scheme $\text{PKE}_C = (\text{KeyGen}, \text{Enc}_C, \text{Dec})$ satisfies δ -correctness or a security definition (IND-CPA, OW-CPA, etc.) if its associated PKE scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ does.*

We emphasize that in our setting, confirmation codes are *not* made public, but will be directly consumed within the key derivation. Security for confirmation-code-augmented PKE accordingly does not (need to) expose confirmation codes for our treatment. In other settings, one may consider a stronger variant where the confirmation code is given to the adversary. For ML-KEM and HQC, there is strong evidence that the underlying PKE remains secure under such variant (cf. the discussion in Section 5), but this stronger security model is outside of the scope of this work.

CONFIRMATION CODE UNPREDICTABILITY (cUP) The idea of confirmation codes is that it should be unlikely that a flawed implementation produces them by accident. Inspired by the “verifiable verification” methodology of Fischlin and Günther [9], we formalize this property as *confirmation code UnPredictability* (cUP), in a technically revised form.

Intuitively, our notion reflects the probability that a faulty implementation of FO re-encryption (treated as adversary $\mathcal{A}$) produces the same confirmation code as the original encryption call. Formalizing this intuition turns out to be challenging. A first straw-man proposal for a cUP definition may be to ask the adversary $\mathcal{A}$ to compute the confirmation code that a fresh run of Enc_C would have produced, when just seeing the public key and the resulting ciphertext. This definition however permits unsuited confirmation codes: the encrypted message itself, or the randomness used to encrypt it, would be deemed good (“unpredictable”) confirmation codes. Yet, in the FO re-encryption setting we are interested in, both message and randomness are recovered upon decapsulation, and a flawed implementation can easily use them without running re-encryption. The problem is that, similar to the setting of verifiable verification discussed in [9], the FO re-encryption process—by design—has all the inputs to Enc required to compute the correct confirmation code, in particular the public key, message, and randomness. (Arguably, it even has the secret key, given that re-encryption happens

$\text{Exp}_{\text{PKE}_C, F, \text{max}}^{\text{cUP}}(\mathcal{A})$ <pre style="margin: 0;"> 01 (pk, sk) ←\$ KeyGen^F() 02 (m, r) ←\$ M × R 03 (c, cd) ← Enc_C^F(pk, m; r) 04 cd' ← A^F(pk, sk, c, m, r) 05 return [[cd = cd']] F̄(x) //at most max many queries 06 return F(x)</pre>

Fig. 8: Confirmation code unpredictability experiment cUP for PKE_C involving function F . The definition for deterministic schemes simply drops the randomness r in lines 03 and 04.

inside the decapsulation algorithm.) If we were to provide those inputs to the cUP adversary $\mathcal{A}$ without restrictions, $\mathcal{A}$ could trivially compute the correct confirmation code, which would thus render the definition moot.

The question is hence how to restrict an cUP adversary in a way that captures accidentally skipped re-encryption in an FO decapsulation implementation, and that accounts for the fact that $\mathcal{A}$ should have access to all encryption inputs by design. Our proposal, formalized in Fig. 8, provides $\mathcal{A}$ with all these inputs (namely, public and secret key, ciphertext, message, and encryption randomness). The restriction is *which computations* $\mathcal{A}$ may perform; this is the technically novel revision of the unpredictability approach of [9].

More specifically, we abstractly limit $\mathcal{A}$'s access to a function F that is involved in the PKE scheme to certain number max of calls, modeling F as a random oracle. If re-encryption and code computation now involve F , the intuition behind limiting access to F is the following: an erroneous implementation that skips re-encryption would not accidentally compute some inner values resulting from calls to F that happen during that re-encryption step.⁶

For a concrete example, we will later see that the PKE encryption step in ML-KEM samples error vectors using SHAKE as an extendable output function on the encryption randomness and a counter. Modeling SHAKE as a random oracle, this allows us to use the entropy in the error vectors that is lost due to rounding lower-order bits of the ciphertext as confirmation code. A flawed implementation skipping re-encryption would have to accidentally also call SHAKE on the right inputs and compute the uncompressed ciphertexts to still produce the correct confirmation codes. We argue that this approach indeed captures the intuition of unpredictable confirmation codes.

Definition 11 (cUP). Let $\text{PKE}_C^F = (\text{KeyGen}, \text{Enc}_C, \text{Dec})$ be a PKE scheme that is confirmation-code-augmented and involves some function F . We define the cUP experiment for PKE_C , modeling F as a random oracle and parameterized by a number $\text{max} \in \mathbb{N}$ in Fig. 8 and the cUP advantage function of an adversary $\mathcal{A}$ against PKE_C as

$$\text{Adv}_{\text{PKE}_C, F, \text{max}}^{\text{cUP}}(\mathcal{A}) = \Pr[\text{Exp}_{\text{PKE}_C, F, \text{max}}^{\text{cUP}}(\mathcal{A}) \Rightarrow 1] .$$

The experiment limits $\mathcal{A}$'s access to F (via oracle $\bar{F}$) to at most max queries.

⁶ Let us stress once more that we are not interested in capturing *malicious* implementations—which may of course skip re-encryption but make targeted calls to F —but in capturing accidental implementation errors.

$$\begin{array}{l} \text{Enc}'_C(\text{pk}, m) \\ 07 \ (c, \text{cd}) \leftarrow \text{Enc}_C(\text{pk}, m; G'(m)) \\ 08 \ \text{return } (c, \text{cd}) \end{array}$$

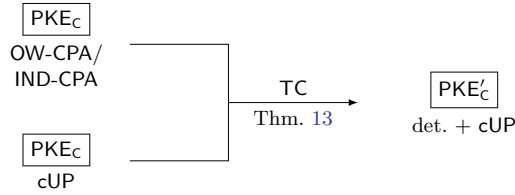
Fig. 9: Derandomized code-augmented encryption algorithm Enc'_C of $\text{TC}[\text{PKE}_C, G']$

3.2 cUP Security of the Confirmation-augmented T-Transform

Notably, the FO PKE-to-KEM transformation requires a deterministic PKE scheme. To enable IND-CCA-secure KEMs based on probabilistic PKE schemes, the underlying PKE scheme first gets derandomized, which is formalized via the T-transform.

$$\begin{array}{ccc} \boxed{\text{PKE}} & \xrightarrow[\text{FO}_m^\perp = U_m^\perp \circ T]{\text{FO}_m^\perp = U_m^\perp \circ T} & \boxed{\text{KEM}} \\ \text{IND-CPA/} & & \text{IND-CCA} \\ \text{OW-CPA} & & \end{array}$$

To enable our augmentation approach for probabilistic PKE schemes, one would thus have to design a confirmation code for the T-derandomized scheme. Designing a code for the T-derandomized scheme $\text{T}[\text{PKE}]$ and justifying its unpredictability might be an annoying task, given that $\text{T}[\text{PKE}]$ is usually analyzed in the (quantum) ROM. It might be much preferable if one could simply design a code for the underlying probabilistic scheme. To simplify the design process, we thus develop a confirmation-code-augmented counterpart TC to the T-transform that behaves exactly like T, except that it additionally turns the underlying confirmation code into one that is suitable for $\text{T}[\text{PKE}]$. We then show that TC maintains code unpredictability.



TC is called TC because it resembles the derandomization transformation T from [14], except that it derandomizes a code-augmented encryption algorithm Enc_C instead of a plain encryption algorithm Enc , and except that it does not introduce a re-encryption check during decryption.

Definition 12 (Transformation TC). Let $\text{PKE}_C = (\text{KeyGen}, \text{Enc}_C, \text{Dec})$ be a confirmation-code-augmented PKE scheme with message space $\mathcal{M}$ and encryption randomness space $\mathcal{R}$, and let $G' : \mathcal{M} \rightarrow \mathcal{R}$ be a hash function. To PKE_C and a hash function $G' : \mathcal{M} \rightarrow \mathcal{R}$, we associate

$$\text{TC}[\text{PKE}_C, G'] = (\text{KeyGen}, \text{Enc}'_C, \text{Dec}) .$$

Algorithm Enc'_C is given in Fig. 9.

CODE UNPREDICTABILITY OF TC IN THE ROM. Intuitively, the following theorem states that TC maintains code unpredictability, provided the underlying scheme is passively secure. We do not consider quantum attackers: code unpredictability is just a stepping stone towards our ultimate goal, noticeable correctness errors in faulty implementations (see next section). The intuition for cUP thus is likewise that $\mathcal{A}$ models a “faulty-but-benign” implementation, and such implementations are unlikely to use a quantum computer. (We note, however, that the result could also be obtained in the quantum-accessible ROM.)

Theorem 13 (PKE_C cUP and PKE OW-CPA/IND-CPA $\xrightarrow{\text{ROM}}$ $\text{TC}[\text{PKE}, G']$ cUP). *Let $\text{PKE}_C = (\text{KeyGen}, \text{Enc}_C, \text{Dec})$ be a confirmation-code-augmented PKE scheme, and let $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ denote its associated plain PKE scheme. For any cUP adversary $\mathcal{A}$ against $\text{TC}[\text{PKE}_C, G']$, where we model G' as a random oracle, there exists a OW-CPA adversary $\mathcal{B}$ against $\text{T}[\text{PKE}, G']$ such that*

$$\text{Adv}_{\text{TC}[\text{PKE}_C, G']}^{\text{cUP}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}_C}^{\text{cUP}}(\mathcal{A}) + \text{Adv}_{\text{T}[\text{PKE}, G']}^{\text{OW-CPA}}(\mathcal{B}) ,$$

and according to [14], there exists a OW-CPA adversary $\mathcal{C}_O$ and an IND-CPA adversary $\mathcal{C}_I$ against PKE such that

$$\text{Adv}_{\text{T}[\text{PKE}, G']}^{\text{OW-CPA}}(\mathcal{B}) \leq \begin{cases} (q_G + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{C}_O) \\ 3 \cdot \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{C}_I) + \frac{2q_G + 1}{|\mathcal{M}|} \end{cases} ,$$

where q_G denotes the number of queries made by $\mathcal{A}$ to the random oracle G' . The running time of $\mathcal{B}$, $\mathcal{C}_O$ and $\mathcal{C}_I$ is about that of $\mathcal{A}$.

To a cUP adversary $\mathcal{A}$ trying to predict the code belonging to a ciphertext, there is not too much of a difference between attacking PKE_C or its derandomized version $\text{TC}[\text{PKE}_C, G']$: in the random oracle model, a challenge taken from $\text{TC}[\text{PKE}_C, G']$ only differs from a challenge taken from PKE_C if $\mathcal{A}$ queries G' on the challenge plaintext, thereby breaking OW-CPA security of $\text{T}[\text{PKE}, G']$. The probability of breaking $\text{T}[\text{PKE}, G']$ was already analyzed in [14, Thms. 3.1, 3.2], we recall these results in Appendix A for convenience. For completeness, we also formally prove Theorem 13 in Appendix B.

4 Confirmation-code-augmented FO-Transform

In this section, we show how to slightly modify FO-like transforms in a way such that implementations can no longer accidentally skip the re-encryption step without triggering functionality issues. We start with formalizing the property we want to achieve, faulty implementation correctness (fCOR), in Section 4.1. We then present our modified FO-like transforms for deterministic schemes in Section 4.2, and show that they achieve the fCOR property. (We also show that our augmentation approach does not degrade security: the modified transforms still achieve IND-CCA security with security bounds that are similar to their non-modified predecessors.) Lastly, we do the same for probabilistic schemes in Section 4.3.

4.1 fCOR: Formalizing that Faulty Implementations Are Noticed

We start by capturing the notion that an implementation cannot accidentally skip the re-encryption step without triggering functionality issues. To that end, we define generic **F**aulty implementation **C**ORrectness (fCOR) for KEMs below, see the left of Fig. 10. In the fCOR game, we model the faulty-but-benign decapsulation implementation as an adversary $\mathcal{A}$, whose goal is to output the correct shared secret matching that of an honest encapsulation (c, K_0) sampled according to $\text{KEM.Encaps}(\text{pk})$. The adversary $\mathcal{A}$ is given pk , c , and also sk —clearly, without any restrictions, $\mathcal{A}$ can trivially compute K_0 by computing $\text{KEM.Decaps}(\text{sk}, c)$. As for unpredictability, we restrict $\mathcal{A}$'s access to a function F used within the KEM algorithms and modeled as a random oracle, to at most max queries. The intuition here is that F is essential for some part of the KEM's decapsulation algorithm, and limiting access to F means that the faulty-but-benign implementation does not fully (or, correctly) implement decapsulation.

Our fCOR notion is generic and applies to any confirmation-code-augmented KEM. Here, we are of course interested in FO-transformed, confirmation-code-augmented KEMs resulting from the $\text{UC}_m^\perp$ and $\text{UC}_m^\perp$ transforms. On the right-hand side of Fig. 10, we thus inline a $\text{UC}_m^\perp$ -FO-transformed

$\text{Exp}_{\text{KEM}, \text{F}, \text{max}}^{\text{fCOR}}(\mathcal{A})$	$\text{Exp}_{\text{KEM}_C^{\mathcal{F}}, \text{F}, \text{max}}^{\text{fCOR}}(\mathcal{A})$
01 $(\text{pk}, \text{sk}) \leftarrow \text{KEM.KeyGen}^{\text{O}, \text{F}}()$	09 $(\text{pk}, (\text{sk}, h, z)) \leftarrow \text{KEM}_C^{\mathcal{F}}.\text{KeyGen}^{\text{F}}()$
02 $(c, K_0) \leftarrow \text{KEM.Encaps}^{\text{O}, \text{F}}(\text{pk})$	$\quad // \text{KEM}_C^{\mathcal{F}}.\text{Encaps}^{\text{F}}(\text{pk})$
03 $K'_0 \leftarrow \text{KEM.Decaps}^{\text{O}, \text{F}}(\text{sk}, c)$	10 $m \leftarrow \mathcal{M}$
04 if $K'_0 \neq K_0$ $//$ correctness error	11 $(c, \text{cd}_0) \leftarrow \text{PKE}_C.\text{Enc}_C^{\text{F}}(\text{pk}, m)$
05 return 1 $//$ is a trivial win	12 $K_0 \leftarrow G(m, H(\text{pk}), \text{cd}_0)$
06 $K_1 \leftarrow \mathcal{A}^{\text{O}, \bar{\text{F}}}(\text{pk}, \text{sk}, c)$	$\quad // \text{KEM}_C^{\mathcal{F}}.\text{Decaps}^{\text{F}}(\text{sk}, c)$
07 return $\llbracket K_0 = K_1 \rrbracket$	13 $m' \leftarrow \text{PKE}_C.\text{Dec}(\text{sk}, c)$
$\bar{\text{F}}(x)$ $//$ at most max many queries	14 $(c', \text{cd}'_0) \leftarrow \text{PKE}_C.\text{Enc}_C^{\text{F}}(\text{pk}, m')$
08 return $\text{F}(x)$	15 if $m' = \perp$ or $c' \neq c$
	16 $K'_0 \leftarrow J(z, c)$
	17 else
	18 $K'_0 \leftarrow G(m', H(\text{pk}), \text{cd}'_0)$
	19 if $K'_0 \neq K_0$ $//$ correctness error
	20 return 1 $//$ is a trivial win
	21 $K_1 \leftarrow \mathcal{A}^{\bar{\text{F}}}(\text{pk}, \text{sk}, c)$
	22 return $\llbracket K_0 = K_1 \rrbracket$

Fig. 10: Left: Faulty implementation correctness experiment fCOR for a generic KEM KEM using random oracle(s) O and F. Right: Faulty implementation correctness experiment fCOR for a $\text{UC}_m^{\mathcal{F}}$ -FO-transformed, confirmation-code augmented KEM $\text{KEM}_C^{\mathcal{F}}$. $\text{KEM}_C^{\mathcal{F}}$ uses a deterministic confirmation-code-augmented PKE scheme PKE_C (using random oracle F) and hash functions G, H, and J. (The encapsulation and decapsulation code of KEM_C are inlined). The version for a UC_m -FO-transformed, confirmation-code augmented KEM $\text{KEM}_C^{\perp}$ can be derived analogously. The adversary $\mathcal{A}$ has unlimited access to O but is allowed at most max queries to F (via the $\bar{\text{F}}$ oracle).

$\text{KEM}_C^{\mathcal{F}}$ for illustration (the case of $\text{UC}_m^{\perp}$ is analogous). Inspecting the code corresponding to decapsulation, $\text{KEM}_C^{\mathcal{F}}.\text{Decaps}^{\text{F}}(\text{sk}, c)$, shows that limiting $\mathcal{A}$'s access to F translates to $\mathcal{A}$ —representing the faulty-but-benign implementation—not being able to fully implement the re-encryption step. Put differently: for a $\text{UC}_m^{\mathcal{F}}$ -FO-transformed KEM, our fCOR game models that re-encryption is faultily implemented. (Recall again that we want to catch faulty (but benign) implementations; a malicious implementation might of course implement re-encryption and its code computation correctly and still accept any ciphertext.)

The probability of $\mathcal{A}$ winning the fCOR game measures the chance of a faulty-but-benign implementation passing the most basic correctness (e.g., interoperability) test: testing whether a generated ciphertext encapsulating a shared secret decapsulates to the same secret. In contrast to “classical” security advantage notions in cryptography, we are okay with “high” faulty correctness advantages of $\mathcal{A}$, e.g., $1/2$ or $2/3$: as long as there is no adversary that has “very high” advantage close to 1. Assuming basic correctness and interoperability testing runs at least a few iterations, the faulty implementation will exhibit *noticeable* correctness issues in practice.

Definition 14 (Faulty implementation correctness (fCOR)). *Let PKE_C be a deterministic confirmation-code-augmented PKE scheme involving some function F, let G, H, and J be hash functions, and let KEM_C be $\text{KEM}_C^{\perp} = \text{UC}_m^{\perp}[\text{PKE}_C, G, H, J]$ or $\text{KEM}_C^{\mathcal{F}} = \text{UC}_m^{\mathcal{F}}[\text{PKE}_C, G, H, J]$ from Definition 15.*

KeyGen^χ	$\text{Encaps}_C(\text{pk})$	$\text{Decaps}_C^\chi(\text{sk}' = (\text{sk}, h, z), c)$
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$	05 $m \leftarrow \mathcal{M}$	09 $m' \leftarrow \text{Dec}(\text{sk}, c)$
02 $h \leftarrow \text{H}(\text{pk}) ; z \leftarrow \mathcal{M}$	06 $(c, \text{cd}) \leftarrow \text{Enc}_C(\text{pk}, m)$	10 $(c', \text{cd}') \leftarrow \text{Enc}_C(\text{pk}, m')$
03 $\text{sk}' \leftarrow (\text{sk}, h, z)$	07 $K \leftarrow \text{G}(m, \text{H}(\text{pk}), \text{cd})$	11 $K' \leftarrow \text{G}(m', h, \text{cd}')$
04 return (pk, sk')	08 return (K, c)	12 $\bar{K} \leftarrow \text{J}(z, c)$
		13 if $m' = \perp$ or $c' \neq c$
		14 return $\bar{K}$
		15 else return K'

Fig. 11: Algorithms of the *implicitly rejecting* $\text{KEM}_C^\chi = (\text{KeyGen}^\chi, \text{Encaps}_C, \text{Decaps}_C^\chi)$, built via the confirmation-augmented counterpart UC_m^χ to the transform used by ML-KEM. We highlight the differences between ML-KEM's transform and UC_m^χ in **violet boxes**. As discussed in the text, we decided to call our transformation UC_m^χ because it is similar in spirit to transformation U_m^χ .

We define the fault implementation correctness (fCOR) experiment for KEM_C using F and parameterized by a number $\text{max} \in \mathbb{N}$ in Fig. 10 and the fCOR advantage function of an adversary $\mathcal{A}$ against KEM_C as

$$\text{Adv}_{\text{KEM}_C, \text{F}, \text{max}}^{\text{fCOR}}(\mathcal{A}) = \Pr[\text{Exp}_{\text{KEM}_C, \text{F}, \text{max}}^{\text{fCOR}}(\mathcal{A}) \Rightarrow 1] .$$

4.2 UC_m^χ and $\text{UC}_m^\perp$: Confirmation-code-augmented FO Constructions for Deterministic PKE Schemes

We start by presenting our modified FO-like transform for *deterministic* schemes: we augment the original U_m -transforms for deterministic schemes from [14] by

- including a confirmation code into the key derivation procedure to achieve faulty implementation correctness; and additionally,
- incorporating public-key identifiers, which reflects recent designs that target multi-user security;
- incorporating a re-encryption check, to prevent issues connected to rigidity [5].

We first define our augmented transformations $\text{UC}_m^\perp$ (rejecting explicitly) and UC_m^χ (rejecting implicitly).

Definition 15 (Confirmation-augmented U_m -transforms UC_m^χ and $\text{UC}_m^\perp$). To a deterministic confirmation-code-augmented public-key encryption scheme $\text{PKE}_C = (\text{KeyGen}, \text{Enc}_C, \text{Dec})$ and hash functions G , H , and J , we associate

$$\text{KEM}_C^\chi = \text{UC}_m^\chi[\text{PKE}_C, \text{G}, \text{H}, \text{J}] = (\text{KeyGen}^\chi, \text{Encaps}, \text{Decaps}^\chi)$$

and

$$\text{KEM}_C^\perp = \text{UC}_m^\perp[\text{PKE}_C, \text{G}, \text{H}, \text{J}] = (\text{KeyGen}, \text{Encaps}, \text{Decaps}^\perp) ,$$

where the respective algorithms are defined in Fig. 11 and Fig. 12.

MOTIVATION OF CONSTRUCTION. The purpose of including cd in G 's input is to prevent an implementation which accidentally skips the re-encryption check from still agreeing with the sender on the shared secret K , assuming that cd was suitably chosen.

$\text{KeyGen}^\perp$	$\text{Encaps}_c(\text{pk})$	$\text{Decaps}_c^\perp(\text{sk}' = (\text{sk}, h), c)$
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$	05 $m \leftarrow \mathcal{M}$	09 $m' \leftarrow \text{Dec}(\text{sk}, c)$
02 $h \leftarrow H(\text{pk})$	06 $(c, \text{cd}) \leftarrow \text{Enc}_c(\text{pk}, m)$	10 $(c', \text{cd}') \leftarrow \text{Enc}_c(\text{pk}, m')$
03 $\text{sk}' \leftarrow (\text{sk}, h)$	07 $K \leftarrow G(m, H(\text{pk}), \text{cd})$	11 $K' \leftarrow G(m', h, \text{cd}')$
04 return (pk, sk')	08 return (K, c)	12 if $m' = \perp$ or $c' \neq c$
		13 return $\perp$
		14 else return K'

Fig. 12: Algorithms of the *explicitly rejecting* $\text{KEM}_c^\perp = (\text{KeyGen}^\perp, \text{Encaps}_c, \text{Decaps}_c^\perp)$, built via our confirmation-code-augmented $\text{UC}_m^\perp$ transform. Again, as discussed, we call our transformation $\text{UC}_m^\perp$ because it is similar in spirit to transformation $\text{U}_m^\perp$. We highlight the the code augmentation in violet boxes .

We now show that the confirmation-code-augmented U_m -transform indeed achieves our goal: assuming that the confirmation codes are sufficiently unpredictable, we show that if the corresponding transformed KEM $\text{KEM}_c^\perp = \text{UC}_m^\perp[\text{PKE}_c, G, H, J]$ or $\text{KEM}_c^\perp = \text{UC}_m^\perp[\text{PKE}_c, G, H, J]$ is faultily implemented and does not perform all F calls necessary for the re-encryption check, it will have noticeable correctness errors. Intuitively, such an implementation will not be able to produce the proper code and thus not manage to produce the right session key, unless it finds a suitable collision in G.

Theorem 16 ($\text{UC}_m^\perp$ and $\text{UC}_m^\perp$: cUP $\implies$ fCOR). *A UC_m -transformed KEM from a PKE scheme with confirmation-code unpredictability is noticeably incorrect under faulty implementation, when modeling the key derivation function G as random oracle.*

Concretely, let PKE_c be an arbitrary confirmation-augmented PKE scheme that is δ -correct. For any adversary $\mathcal{A}$ against the faulty implementation correctness game for KEM_c being $\text{KEM}_c^\perp = \text{UC}_m^\perp[\text{PKE}_c, G, H, J]$ or $\text{KEM}_c^\perp = \text{UC}_m^\perp[\text{PKE}_c, G, H, J]$ where we model G as an (observable, non-programmable) random oracle with output length ℓ_G , function F, and parameter max, we construct an adversary $\mathcal{B}$ against the unpredictability (cUP) of PKE_c , such that

$$\text{Adv}_{\text{KEM}_c, \text{F}, \text{max}}^{\text{fCOR}}(\mathcal{A}) \leq \delta + \text{Adv}_{\text{PKE}_c, \text{F}, \text{max}}^{\text{cUP}}(\mathcal{B}) + 2^{-\ell_G}.$$

For the short confirmation codes we propose for ML-KEM and HQC (cf. Section 5), the cUP advantage is essentially also the fCOR advantage for the resulting KEMs, as the former dominates the bound above.

Proof. The proof applies identically to $\text{KEM}_c^\perp$ and $\text{KEM}_c^\perp$.

Let $(c, \text{cd}_0) = \text{PKE}_c.\text{Enc}_c^F(\text{pk}, m)$, $K_0 = G(m, \text{cd}_0, H(\text{pk}))$, $m' = \text{PKE}_c.\text{Dec}(\text{sk}, c)$, and $K'_0 = J(z, c)$ or $K'_0 = G(m', \text{cd}'_0, H(\text{pk}))$ denote the values that were computed by the fCOR game (expanded for $\text{KEM}_c^\perp$) in lines 01–18 of Fig. 10, and let K_1 be the shared secret that was output by $\mathcal{A}$. For $\mathcal{A}$ to win, we must have that (a) $K'_0 \neq K_0$ or (b) $K_0 = G(m, \text{cd}_0, H(\text{pk})) = K_1$. We note that (a) constitutes a correctness error, so we can upper-bound the probability of (a) happening by δ . For (b), we distinguish two cases:

- Case 1: $\mathcal{A}$ queries $(m, \text{cd}_0, H(\text{pk}))$ to the random oracle G. This case induces an adversary $\mathcal{B}$ against $\text{Exp}_{\text{PKE}_c, \text{F}, \text{max}}^{\text{cUP}}$ that simulates $\bar{F}$ by relaying to its own oracle, observes the queries $\mathcal{A}$ makes to G, and upon the matching query $(m, \text{cd}_0, H(\text{pk}))$ outputs cd_0 .
- Case 2: $\mathcal{A}$ does not query $(m, \text{cd}_0, H(\text{pk}))$ to the random oracle G. In that case, $K_0 = G(m, \text{cd}_0, H(\text{pk}))$ to $\mathcal{A}$ is an unknown random value of length ℓ_G . So $\mathcal{A}$'s chance of guessing it with its output K_1 is $1/2^{\ell_G}$. $\square$

Confirmation-code-augmentation does not degrade security. We conclude this subsection by discussing that our augmentation approach does not degrade security: we show that the augmented transform $UC_m^\perp$ still achieves IND-CCA security. We analyze transformation $UC_m^\perp$ in ‘standalone’ mode, i.e., when $UC_m^\perp$ is applied to a generic *deterministic* PKE scheme that is confirmation-augmented with *deterministic* code generation. We also discuss the security of other variants of these transformations (Remark 17).

INTUITION WHY $UC_m^\perp$ IS SECURE WHEN APPLIED TO DETERMINISTIC SCHEMES. Our transform resembles transformation $U_m^\perp$ that was first analyzed in [14, Thm. 3.6]. The only differences between $UC_m^\perp$ and $U_m^\perp$ are that when deriving session keys, $UC_m^\perp$ takes two additional inputs to $G - H(pk)$ and cd – and that $UC_m^\perp$ performs a re-encryption check. The re-encryption check prevents certain CCA attacks (and thus only makes the transform stronger). The additional input $H(pk)$ is a value that everybody can compute, it thus does not weaken the resulting session key. While the additional input cd does depend on its originating message m , this dependence is deterministic by our requirement on Enc_C . It thus creates no additional information/attack surfaces when being hashed together with m .

We make this intuition formal in Appendix C (for the ROM, see Thm. 26 in Appendix C.1, for the QROM, see Thm. 27 in Appendix C.2). We defer to the appendix because these proofs essentially redo previous proofs for non-augmented U-transformations, in the ROM [15, Thm. 2.1.7] and the QROM [6, Thm. 2].

$$\boxed{\text{PKE}'}_{\text{det} + \text{OW-CPA}} \xrightarrow[\text{ROM Thm. 26/QROM Thm. 27}]{UC_m^\perp} \boxed{\text{KEM}_C}_{\text{IND-CCA}}$$

Remark 17 (IND-CCA security for other code-augmented U-variants). One can obtain a proof for the explicitly rejecting variant $UC_m^\perp$ that recovers the ROM bound [14, Thm. 3.5] for $U_m^\perp$, using the same reasoning as in the ROM proof for $UC_m^\perp$. The QROM situation of variants with explicit rejection mode so far is undetermined; we are not aware of any QROM result for standalone U-transforms in explicit mode (without key confirmation) at the time of writing.

We also remark that the results for $UC_m^\perp$ can be recovered in a straightforward manner for variants of $UC_m^\perp$ that

- omit $H(pk)$ from the hash input since the hashing of pk has absolutely no influence on (single-instance) IND-CCA security;
- include the ciphertext c into the hash input [6, Thm. 5].

4.3 $FOC_m^\perp/FOC_m^\perp$: Confirmation-code-augmented FO Constructions for Probabilistic PKE Schemes

We now present our transformations for *probabilistic* schemes.

Definition 18 (Confirmation-augmented transforms $FOC_m^\perp/FOC_m^\perp$). Transformations $FOC_m^\perp/FOC_m^\perp$ are the respective transformation $UC_m^\perp/UC_m^\perp$, used in conjunction with TC. In other words, the construction is exactly like the construction of $UC_m^\perp/UC_m^\perp$ (see Fig. 12), except that it derandomizes encryption (in lines 06 and 10). Encapsulation and Decapsulation do not sample the randomness for encryption at random, but instead derive it as $G'(m)$ resp. $G'(m')$.

Next, we show that the confirmation-code-augmented FO-transforms indeed achieve our goal, fCOR. This result is a corollary, as it immediately follows from combining the fCOR result for the ‘standalone’ transformation $UC_m^\perp/UC_m^\perp$ (Theorem 16) with the result that TC maintains code unpredictability (Theorem 13).

Corollary 19. *A FOC_m^χ -transformed KEM from a probabilistic PKE scheme with confirmation-code unpredictability is noticeably incorrect under faulty implementation, when modeling the key derivation function G as random oracle.*

Concretely, let PKE_C be an arbitrary confirmation-augmented PKE scheme that is δ -correct. For any adversary $\mathcal{A}$ against the faulty implementation correctness game for KEM_C being $\text{KEM}_C^\chi = \text{FOC}_m^\chi[\text{PKE}_C, G, H, J, G']$ or $\text{KEM}_C^\perp = \text{FOC}_m^\perp[\text{PKE}_C, G, H, J, G']$, where we model G as an (observable, non-programmable) random oracle with output length ℓ_G , function F , and parameter max , we construct an adversary $\mathcal{B}$ against the unpredictability (cUP) of PKE_C , such that

$$\text{Adv}_{\text{KEM}_C, F, \text{max}}^{\text{fCOR}}(\mathcal{A}) \leq \delta + \text{Adv}_{\text{PKE}_C}^{\text{cUP}}(\mathcal{A}) + \text{Adv}_{\text{TC}[\text{PKE}_C, G']}^{\text{OW-CPA}}(\mathcal{B}) + 2^{-\ell_G},$$

and according to [14], there exists a OW-CPA adversary $\mathcal{C}_O$ and an IND-CPA adversary $\mathcal{C}_I$ against PKE such that

$$\text{Adv}_{\text{TC}[\text{PKE}_C, G']}^{\text{OW-CPA}}(\mathcal{B}) \leq \begin{cases} (q_G + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{C}_O) \\ 3 \cdot \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{C}_I) + \frac{2q_G + 1}{|\mathcal{M}|} \end{cases},$$

where q_G denotes the number of queries made by $\mathcal{A}$ to the random oracle modeling G' . The running time of $\mathcal{B}$, $\mathcal{C}_O$ and $\mathcal{C}_I$ is about that of $\mathcal{A}$.

Confirmation-code-augmentation does not degrade security. We again conclude this subsection by discussing that our augmentation approach does not degrade security: we show that the augmented transform FOC_m^χ still achieves IND-CCA security. We also discuss the security of other variants of these transformations (Remark 20).

INTUITION WHY FOC_m^χ IS SECURE. Our transform resembles the FO_m^χ transform that was first analyzed in [14]. The only differences are that when deriving session keys, FOC_m^χ takes two additional inputs to G – $H(\text{pk})$ and cd . Again, the additional input $H(\text{pk})$ is a value that everybody can compute and thus does not weaken the resulting session key. While the additional input cd does depend on its originating message m , this dependence is deterministic since FOC_m^χ derandomizes Enc_C . It thus creates no additional information/attack surfaces when being hashed together with m .

We make this intuition formal in Appendix C (for the ROM, see Cor. 29 in Appendix C.3, for the QROM, see Thm. 30 in Appendix C.4). We again defer to the appendix because Cor. 29 is obtained by combining the UC_m^χ result, Theorem 26, with previous results about T that were given in [14], and QROM Thm. 30 essentially redoes a previous proof for the non-augmented FO_m -transformations [16].

$$\begin{array}{ccc} \boxed{\text{PKE}} & \xrightarrow[\text{ROM Cor. 29, QROM Thm. 30}]{\text{FOC}_m^\chi = \text{UC}_m^\chi \circ \text{TC}} & \boxed{\text{KEM}_C} \\ \text{OW-CPA/} & & \text{IND-CCA} \\ \text{IND-CPA} & & \end{array}$$

Remark 20 (IND-CCA security for other code-augmented FO-variants). In the previous subsection, we also discussed (see Remark 17) the security of other variants of our construction for deterministic schemes, UC. Unlike for $\text{UC}_m^\perp$, however, transformation $\text{FO}_m^\perp$ is known to have a QROM security proof [16] that achieves the same bound as FO_m^χ . They essentially have the same proof, one can thus also obtain a QROM bound for $\text{FOC}_m^\perp$.

The remarks about alternative hash inputs also apply for our construction for probabilistic schemes, FOC.

5 Application to Post-Quantum KEMs

We now turn to augmenting real-world post-quantum KEM schemes with confirmation codes that are unpredictable and enable catching faulty implementations of decapsulation through noticeable correctness errors. We discuss and analyze proposals for both ML-KEM, standardized by NIST in FIPS 203 [23], as well as HQC [2], a Round 4 submission to NIST’s PQC standardization process, which was selected by NIST for standardization in 2025.

5.1 ML-KEM

Recall that ML-KEM operates on elements of $R_q = \mathbb{Z}_{3329}[X]/(X^{256} + 1)$. In the PKE scheme underlying ML-KEM, the encryption process pseudorandomly samples error terms $\mathbf{e}_1 \in R_q^k$ and $e_2 \in R_q$ (for $k \in \{2, 3, 4\}$ depending on the security level). These error terms are used to compute a vector $\mathbf{u} \in R_q^k$ of polynomials and a polynomial $v \in R_q$. The polynomials in $\mathbf{u}$ and v are then compressed into the final ciphertext (c_1, c_2) . This compression entails a loss of information by rounding coefficients modulo $q = 3329$ to a smaller range, namely modulo 2^{10} or 2^{11} (for $\mathbf{u}$) or 2^4 or 2^5 (for v) (depending on the security level). We leverage this information loss for our confirmation-code-augmented version of ML-KEM, denoted ML-KEM_C. In doing so, we will model the PRF used to derive the error terms as the constrained function F in the cUP security notion.

Recall that confirmation codes are *not* revealed outside our confirmation-code-augmented FO transform, but directly consumed within the key derivation; see also Section 4.2 for the discussion why our confirmation-code-augmented FO transform does not degrade security. We nevertheless note that [7, Theorem 2] shows IND-CPA security of uncompressed Kyber PKE scheme (underlying ML-KEM) under decision-MLWE. (Their Theorem 2 is technically about Kyber PKE with uncompressed public keys and compressed ciphertexts, but the proof arguments actually show IND-CPA security of a variant with uncompressed ciphertexts.)

ADDING CONFIRMATION CODES TO ML-KEM’S PKE. First, we specify how to augment the PKE encryption algorithm of ML-KEM with a confirmation code; the resulting encryption algorithm ML-KEM_C.PKE_C.Enc_C is given in Fig. 13 (the other PKE algorithms remain unchanged). More specifically, parameterized by $S \subseteq \{1, \dots, 256\}$, ML-KEM_C.PKE_C[S].Enc_C outputs as confirmation code cd the coefficients at positions in S of each term of the uncompressed ciphertext values $\mathbf{u} \in R_q^k$ and $v \in R_q$ (see Fig. 13, line 24).

Using a small set S helps reduce the overhead of including cd in the KEM key derivation. We show in the following theorem that a very small set S —just a few coefficients from each uncompressed ciphertext term—already suffices for ML-KEM_C.PKE_C[S] to achieve meaningful confirmation-code unpredictability (cUP). Recall that we do not need cryptographically small advantages for confirmation-code unpredictability: a cUP advantage bound noticeably below 1 already suffices to ensure that a KEM built via the confirmation-code-augmented FO-transforms in Section 4 has noticeable correctness errors upon faulty implementation. For ML-KEM_C.PKE_C[S], we already obtain a cUP advantage upper bound of $\approx 2/3$ for $|S| = 2$ and $\max = k$ (i.e., missing only one of the $k + 1$ error polynomials in $\mathbf{e}_1 \in R_q^k, e_2 \in R_q$). Note that we allow parameterizing ML-KEM_C.PKE_C[S] with a set of coefficient positions for generality; in practice, slicing off a consecutive sequence of coefficients at the start or end of each polynomial is sufficiently efficient, as our performance evaluation in Section 5.3 shows.

USING CONFIRMATION CODES IN ML-KEM’S ENCAPSULATION/DECAPSULATION. We now explain how to integrate the confirmation-code-augmented PKE scheme into the encapsulation and decapsulation algorithms of ML-KEM to obtain our confirmation-code-augmented version ML-KEM_C.

As shown in Fig. 14, ML-KEM uses a variant of the $\text{FO}_m^x = \text{U}_m^x \circ \text{T}$ transform, deriving randomness and shared secret jointly as $(K, r) \leftarrow \text{SHA3-512}(m \parallel \text{SHA3-256}(pk))$ before re-encryption. Applying

```

ML-KEMC.PKEC[S].EncC( $pk \in \mathcal{B}^{12 \cdot k \cdot n/8+32}, m \in \mathcal{B}^{32}, r \in \mathcal{B}^{32}$ )
01  $N \leftarrow 0$ 
02 parse  $\hat{\mathbf{t}}$  from  $pk$ 
03 parse  $\rho$  from  $pk$ 
04  $\hat{\mathbf{A}} \leftarrow$  (pseudorandomly generate matrix  $\hat{\mathbf{A}} \in (\mathbb{Z}_q^{256})^{k \times k}$  from  $\rho$ )
05  $\mathbf{y} \leftarrow$  (pseudorandomly generate  $\mathbf{y} \in (\mathbb{Z}_q^{256})^k$  from  $r$ ; set  $N \leftarrow k$ )
13 for  $i$  from 0 to  $k - 1$ : //generate  $\mathbf{e}_1 \in (\mathbb{Z}_q^{256})^k$ 
14    $\mathbf{e}_1[i] \leftarrow \text{SamplePolyCBD}_{\eta_2}(\text{PRF}_{\eta_2}(r, N))$ 
15    $N \leftarrow N + 1$ 
16 endfor
17  $\mathbf{e}_2 \leftarrow \text{SamplePolyCBD}_{\eta_2}(\text{PRF}_{\eta_2}(r, N))$  //generate  $\mathbf{e}_2 \in \mathbb{Z}_q^{256}$ 
18  $\hat{\mathbf{y}} \leftarrow \text{NTT}(\mathbf{y})$ 
19  $\mathbf{u} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}}^T \circ \hat{\mathbf{y}}) + \mathbf{e}_1$ 
20  $\mu \leftarrow \text{Decompress}_1(\text{ByteDecode}_1(m))$ 
21  $\mathbf{v} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{t}}^T \circ \hat{\mathbf{y}}) + \mathbf{e}_2 + \mu$ 
22  $\mathbf{c}_1 \leftarrow \text{ByteEncode}_{d_u}(\text{Compress}_{d_u}(\mathbf{u}))$ 
23  $\mathbf{c}_2 \leftarrow \text{ByteEncode}_{d_v}(\text{Compress}_{d_v}(\mathbf{v}))$ 
   //confirmation code:
   //uncompressed ciphertext coefficients at positions  $S \subseteq \{1, \dots, 256\}$  of each term
24  $\mathbf{cd} \leftarrow (\mathbf{u}[1][S], \dots, \mathbf{u}[k][S], \mathbf{v}[S])$ 
25 return ( $c = c_1 \parallel c_2, \mathbf{cd}$ )

```

Fig. 13: Confirmation-code-augmented version $\text{ML-KEM}_C.\text{PKE}_C[S]$ of ML-KEM’s PKE encryption algorithm, with the length of the confirmation code parameterized by $S \subseteq \{1, \dots, 256\}$. Changes introducing confirmation codes are highlighted in **violet boxes**. In our analysis, we model PRF as (extendable-output) random oracle F , where PRF_η denotes a call producing a $(64 \cdot \eta)$ -byte output; for all parameter sets, $\eta_2 = 2$. The line numbering follows the ML-KEM specification in NIST FIPS 203 [23, Algorithm 14, p. 30]. Some details not relevant to this paper are omitted; see the FIPS 203 specification for details.

our confirmation-code-augmented transform $\text{FOC}_m^\chi = \text{UC}_m^\chi \circ \text{TC}$ from Section 4, we separate the derivation of randomness and shared secret, to include the confirmation code in the latter (obtained through re-encryption). We thus replace the single SHA3-512 call with two calls of SHA3-256; see Fig. 14 for the result. Note that it is important to ensure domain separation between these calls of SHA3-256 and other SHA3-256 calls in ML-KEM, such as the hashing of pk , for example by using labels or ensuring inputs are different length [4]; this requires carefully checking the size of the confirmation code relative to the pk size. When the confirmation code is chosen to be shorter than 736 bytes (and non-empty), domain separation is achieved due to distinct input lengths for each call, namely 800–1568 bytes for the call hashing pk , 64 bytes for the call deriving r , and 65–799 bytes for the call deriving K .

SECURITY EVALUATION. The following theorem shows that $\text{ML-KEM}_C.\text{PKE}_C[S]$ achieves confirmation-code unpredictability (cUP), when ML-KEM’s PRF is modeled as a random oracle to which calls by the faulty implementation are constrained.

Theorem 21 (ML-KEM_C.PKE_C is cUP). *Let $\text{ML-KEM}_C.\text{PKE}_C[S]$ be the confirmation-code-augmented PKE scheme of ML-KEM_C from Fig. 13 involving function $F = \text{PRF}$, using uncompressed ciphertext coefficients from the positions $S \subseteq \{1, \dots, 256\}$ as confirmation code. Let $\mathcal{A}$ be an adversary in the*

<pre> ML-KEM.Encaps_internal($pk, m \in \mathcal{B}^{32}$) 01 $(K, r) \leftarrow \text{SHA3-512}(m \parallel \text{SHA3-256}(pk))$ 02 $c \leftarrow \text{ML-KEM.PKE.Enc}(pk, m; r)$ 03 return (K, c) </pre>	<pre> ML-KEM_C.Encaps_internal($pk, m \in \mathcal{B}^{32}$) 04 $r \leftarrow \text{SHA3-256}(m \parallel \text{SHA3-256}(pk))$ 05 $(c, \text{cd}) \leftarrow \text{ML-KEM}_C.\text{PKE}_C.\text{Enc}_C(pk, m; r)$ 06 $K \leftarrow \text{SHA3-256}(m \parallel \text{SHA3-256}(pk) \parallel \text{cd})$ 07 return (K, c) </pre>
<pre> ML-KEM.Decaps_internal($sk' = (sk, h, z), c$) 08 $m' \leftarrow \text{ML-KEM.PKE.Dec}(sk, c)$ 09 $(K', r') \leftarrow \text{SHA3-512}(m' \parallel h)$ 10 $\bar{K} \leftarrow \text{SHAKE256}(z \parallel c, 8 \cdot 32)$ 11 $c' \leftarrow \text{ML-KEM.PKE.Enc}(pk, m'; r')$ 12 if $c \neq c'$ 13 return $\bar{K}$ 14 return K' </pre>	<pre> ML-KEM_C.Decaps_internal($sk' = (sk, h, z), c$) 15 $m' \leftarrow \text{ML-KEM.PKE.Dec}(sk, c)$ 16 $r' \leftarrow \text{SHA3-256}(m' \parallel h)$ 17 $(c', \text{cd}') \leftarrow \text{ML-KEM}_C.\text{PKE}_C.\text{Enc}_C(pk, m'; r')$ 18 $K' \leftarrow \text{SHA3-256}(m' \parallel h \parallel \text{cd}')$ 19 $\bar{K} \leftarrow \text{SHAKE256}(z \parallel c, 8 \cdot 32)$ 20 if $c \neq c'$ 21 return $\bar{K}$ 22 return K' </pre>

Fig. 14: Internal encapsulation (top) and decapsulation (bottom) algorithms of ML-KEM [23, Algorithm 17, p. 33] (left) and of our confirmation-code-augmented version ML-KEM_C following the $\text{FOC}_m^x = \text{UC}_m^x \circ \text{TC}$ transform (right). We highlight the differences between ML-KEM and ML-KEM_C in violet boxes.

cUP experiment making at most max queries to its $\bar{\text{F}}$ oracle. If we make an additional uniformity assumption on certain values in ML-KEM (stated in the proof), then

$$\text{Adv}_{\text{ML-KEM}_C.\text{PKE}_C[S], \text{PRF}, \text{max}}^{\text{cUP}}(\mathcal{A}) \leq 2^{-h_{e_1} \cdot |S| \cdot (k+1-\text{max})},$$

where h_{e_1} is as shown in Table 1. Without the additional uniformity assumption, for ML-KEM-512 and ML-KEM-768, the advantage of $\mathcal{A}$ is upper-bounded by

$$p_{|S|} = \sum_{i=0}^{|S|(k+1-\text{max})} \binom{|S|(k+1-\text{max})}{i} \left(\frac{2}{16}\right)^{|S|(k+1-\text{max})-i} \left(\frac{14}{16}\right)^i \cdot 2^{-h \cdot i}, \quad (1)$$

where $h \approx 0.322$. Calculated values for the bound in each case for short ($|S| = 2$) and full ($|S| = n = 256$) confirmation codes are shown in Table 1.

Proof. We observe that PRF is used $k+1$ times in $\text{ML-KEM}_C.\text{PKE}_C[S].\text{Enc}_C$, to compute $\mathbf{e}_1[1], \dots, \mathbf{e}[k]$ as well as e_2 .⁷ This means for any $\text{max} \leq k$, $\mathcal{A}$ is missing at least one of the $k+1$ error terms.

Let $\mathbf{u} = \text{NTT}^{-1}(\hat{\mathbf{A}}^T \circ \hat{\mathbf{y}}) + \mathbf{e}_1$ and $v = \text{NTT}^{-1}(\hat{\mathbf{t}}^T \circ \hat{\mathbf{y}}) + e_2 + \mu$ be the uncompressed ciphertexts (as computed during challenge encryption) and $c = c_1 \parallel c_2$ the challenge ciphertext. Since $\mathcal{A}$ knows the secret key sk , we conservatively assume it can compute $\text{NTT}^{-1}(\hat{\mathbf{A}}^T \circ \hat{\mathbf{y}})$, $\text{NTT}^{-1}(\hat{\mathbf{t}}^T \circ \hat{\mathbf{y}})$, and μ . Using its $\bar{\text{F}}$ oracle, $\mathcal{A}$ can furthermore compute $\text{max} \leq k$ many of the $k+1$ error vectors $\mathbf{e}_1[1], \dots, \mathbf{e}[k], e_2$. That is, $\mathcal{A}$ knows the compressed ciphertext as well as all components of the uncompressed ciphertext except $k+1-\text{max}$ error vectors. For each of the $k+1-\text{max}$ missing error vectors, $\mathcal{A}$ needs $|S|$ coefficients to construct the complete confirmation code.

⁷ Strictly speaking, PRF is also used to generate $\mathbf{y}$. But since in the subsequent analysis we will conservatively assume $\mathcal{A}$ knows $\mathbf{y}$, we can also assume w.l.o.g. that $\mathcal{A}$ does not spend its limited $\bar{\text{F}}$ queries on computing $\mathbf{y}$.

Table 1: Calculated error coefficient entropy and confirmation code guessing probabilities for ML-KEM. Conservatively assumes the adversary is allowed $\max = k$ (i.e., all but one) evaluations of PRF. For short ($|S| = 2$) codes, the confirmation code guessing probability is essentially also the fCOR advantage upper bound, as the other terms in the bound of Theorem 16 are negligible in comparison.

Scheme	Conditional entropy of $\mathbf{e}_1$ coeff. of $\mathbf{e}_2$ coeff.		Conf. code guessing probability	
	(h_{e_1})	(h_{e_2})	Short ($ S = 2$) (p_2)	Full ($ S = 256$) (p_{256})
<i>Assuming uniform z</i>				
ML-KEM-512	1.2733	2.0188	0.1712	2^{-325}
ML-KEM-768	1.2733	2.0188	0.1712	2^{-325}
ML-KEM-1024	0.6407	2.0069	0.4114	2^{-164}
<i>Worst-case analysis</i>				
ML-KEM-512	0.3219	0.3219	0.6806	2^{-71}
ML-KEM-768	0.3219	0.3219	0.6806	2^{-71}

Each coefficient of either $\mathbf{u}$ or v is of the form $z + e$, where z is some element $\mathbb{Z}_q$ and e is sampled according to the ML-KEM error distribution, which is the following centered binomial distribution: 0 with probability $6/16$, ± 1 each with probability $4/16$, ± 2 each with probability $1/16$.

The core of the problem is to compute the conditional entropy of a single error coefficient e given $z \in \mathbb{Z}_q$ and the compressed value $c = \text{Compress}_d(z + e \bmod q)$, where

$$\text{Compress}_d(x) = \lceil (2^d / 3329) \cdot x \rceil \bmod 2^d,$$

with $d = d_u = 10$ for compressing $\mathbf{u}$ and $d = d_v = 4$ for compressing v in both ML-KEM-512 and ML-KEM-768, and $d_u = 11$ and $d_v = 5$ correspondingly for ML-KEM-1024.

Since z takes on values in $\{0, \dots, 3328\}$ and e takes on values in $\{0, \pm 1, \pm 2\}$, the space of all possibilities can be efficiently explored numerically. We report bounds for two different analyses: an average-case analysis assuming uniform z , and a worst-case analysis not assuming uniform z .

Assuming uniform z . If we were willing to make the assumption that the distribution of z is uniform on $\mathbb{Z}_{3329}$, then it would suffice to compute the entropy of entries of $\mathbf{e}_1$ or $\mathbf{e}_2$ given $(z, c = \text{Compress}_d(z + e \bmod q))$, namely:

$$H(Y|X) = - \sum_{(z,c) \in X, e \in Y} \Pr[X = (z, c), Y = e] \log_2 \frac{\Pr[X = (z, c), Y = e]}{\Pr[X = (z, c)]}.$$

As observed in Table 1, for all ML-KEM parameters, the conditional entropy h_{e_1} in a coefficient of $\mathbf{e}_1$ is less than or equal to the conditional entropy h_{e_2} in a coefficient of $\mathbf{e}_2$, so the optimal adversary would use its limited $\bar{F}$ queries to learn $\mathbf{e}_2$ first, and then any more of $\mathbf{e}_1$ it can. Since a confirmation code using subset S involves $|S|$ coefficients from each error polynomial, of which there are k in $\mathbf{e}_1$ and 1 more in $\mathbf{e}_2$, and these are all independent of each other, the adversary's success probability is at most

$$p_{|S|} = (2^{-h_{e_1}})^{|S| \cdot (k+1-\max)} = 2^{-h_{e_1} \cdot |S| \cdot (k+1-\max)}.$$

The results of calculating this numerically for ML-KEM are shown in the “Assuming uniform z ” section of Table 1, for the conservative case that the adversary is allowed $\max = k$ (i.e., all but one) queries to $\bar{F}$.

Worst-case analysis, not assuming uniform z . Since the cUP security notion assumes that the secret key is known to the adversary, we cannot employ the decisional LWE assumption to justify z being uniform. In practice, z appears quite close to uniform, but we can also obtain looser but still quite acceptable bounds for ML-KEM-512 and ML-KEM-768 without making any assumptions on the distribution of z .

There are certainly some coefficients for which the conditional entropy of the error, and hence the uncompressed ciphertext coefficient, is 0: these are (z, c) combinations for which there is a single way of building that combination. We observe (through exhaustively searching the combination space) that, for ML-KEM-512 and ML-KEM-768, the combinations for which this can occur are only combinations in which the error takes on values ± 2 , and a simplification would be to consider solely combinations build from error terms in $\{-1, 0, 1\}$. Unfortunately, for ML-KEM-1024, these degenerate combinations can occur with any error value, so we do not proceed with this analysis for ML-KEM-1024.

The conditional min-entropy for cases when the error is in $\{-1, 0, 1\}$ is

$$H_{\min}(Y|X) = -\log_2 \max_{(z,c) \in X, e \in \{-1,0,1\}} \Pr[Y = e|X = (z, c)].$$

As seen in Table 1, for ML-KEM-512 and ML-KEM-768, the conditional min-entropy is the same for coefficients of both $\mathbf{e}_1$ and $\mathbf{e}_2$: $h = h_{e_1} = h_{e_2} \approx 0.322$. Hence it does not matter whether an adversary uses their limited $\bar{F}$ queries to learn $\mathbf{e}_1$ or $\mathbf{e}_2$.

Noting that the degenerate case of error ± 2 occurs only 2/16 of the time, we can compute the probability of adversary guessing the confirmation code for the short confirmation code and the full confirmation code. For a confirmation code with subset S , the adversary’s success probability is a weighted sum of the success probability given that i coefficients involve errors in $\{-1, 0, 1\}$ times the probability that i coefficients take on such error values, which is given in Equation (1). The results of calculating this numerically for ML-KEM-512 and ML-KEM-768 are shown in the “Worst-case analysis” section of Table 1, again conservatively assuming that the adversary is allowed $\max = k$ (i.e., all but one) queries to $\bar{F}$. $\square$

EXPORTING CONFIRMATION CODES. Given that ML-KEM is already finalized as a standard in NIST FIPS 203 [23], practitioners might be hesitant to modify internals of ML-KEM as our confirmation-code augmented version above does. An alternative approach could hence be to export the confirmation code as an additional output on the KEM level, and then post-process the shared secret by hashing it with the confirmation code. This approach could lend itself to retrofitting existing implementations of ML-KEM as in FIPS 203.

5.2 HQC

We briefly recall some necessary preliminaries. HQC operates on elements that can be interchangeably represented as bit strings (row vectors) or polynomials in $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$. We denote $\mathcal{R}_w$ as the space of binary strings with length n and Hamming weight w that is, strings with w non-zero entries. For $\mathbf{u}, \mathbf{v} \in \mathcal{R}$, we define the product $\mathbf{z} = \mathbf{u} \cdot \mathbf{v}$ as

$$\mathbf{z}_k = \sum_{i+j \equiv k \pmod n} u_i v_j \quad \text{for } k \in \{0, 1, \dots, n-1\}.$$

We define three helper functions that each take as input a string x and an integer l : $\text{truncate}(x, l)$ returns the truncation of x by l bits, $\text{firstBytes}(x, l)$ returns the first l bytes of x , and $\text{lastBytes}(x, l)$ returns the last l bytes of x .

We make use of Proposition 3.1.1 from [3].

Proposition 22. *Let $\mathbf{x} = (x_0, \dots, x_{n-1})$ be a random vector chosen uniformly among all binary vectors of length n and weight w and let $\mathbf{r} = (r_0, \dots, r_{n-1})$ be a random vector chosen uniformly among all binary vectors of length n and weight w_r and independently of $\mathbf{x}$. Then denoting $\mathbf{z} = \mathbf{x} \cdot \mathbf{r}$ we have that for every $k \in \{0, \dots, n-1\}$ the k th coordinate z_k of $\mathbf{z}$ is Bernoulli-distributed with parameter*

$$\tilde{p} = \Pr[z_k = 1] = \frac{1}{\binom{n}{w} \binom{n}{w_r}} \sum_{\substack{1 \leq \ell \leq \min(w, w_r) \\ \ell \text{ odd}}} \binom{n}{\ell} \binom{n-\ell}{w-\ell} \binom{n-w}{w_r-\ell}.$$

ADDING CONFIRMATION CODES TO HQC’s PKE. We augment the PKE encryption algorithm of HQC with a confirmation code and, in encapsulation and decapsulation, embed it into the KEM key derivation as illustrated in Fig. 15 (the other algorithms remain unchanged). The randomness for encryption is derived from the message $\mathbf{m}$ and further inputs as $\theta \leftarrow \mathcal{G}(\mathbf{m} \parallel \text{firstBytes}(\text{pk}, 32) \parallel \text{salt})$ using SHAKE. Encryption then pseudorandomly generates vectors $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$ of appropriate weight from θ ; this is internally achieved by pseudorandomly generating uniform byte strings using SHAKE and then deriving vectors of the required weights. These vectors are used along with $\text{pk} = (\mathbf{h}, \mathbf{s})$ to compute ciphertext $\mathbf{c} = (\mathbf{u}, \mathbf{v})$ with $\mathbf{u} = (\mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2)$ and $\mathbf{v} = \text{truncate}(\mathbf{m}\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, \ell)$. We model skipping re-encryption as skipping all lines where random values are generated from θ —in other words, the generation of $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$. We use the last byte of $\mathbf{m}\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$ (i.e., the $\mathbf{v}$ part of the ciphertext *before* truncation) as confirmation code, leveraging that the truncation by ℓ bits for the ciphertext, where ℓ depends on the HQC parameter set. We show that these truncated bits cannot be guessed with high probability without knowing $\mathbf{r}_1, \mathbf{r}_2$, or $\mathbf{e}$.

As for ML-KEM, recall that confirmation codes are *not* revealed outside our confirmation-code-augmented FO transform, but directly consumed within the key derivation; see also Section 4.2. We nevertheless note that while IND-CPA security of HQC’s PKE scheme is shown under a truncated decision syndrome decoding problem (“3-DQCS-D-PT” – the T stands for truncation), the HQC specification [1, page 15] states that “whenever the number of truncated positions is very small compared to the block length n , the impact on the security is negligible with respect to the 3-DQCS-D problem.”

Theorem 23. *Let $\text{HQC}_C.\text{PKE}_C$ be the confirmation-code-augmented PKE scheme of HQC using the Enc_C algorithm from Fig. 15 involving function $F = \text{PRF}$. Let $\mathcal{A}$ be an adversary in the cUP experiment making no ($\max = 0$) queries to its $\bar{F}$ oracle. Then, using the heuristic assumption that the coordinates of $\mathbf{e}$ are independent, as was done in [2, 3], we obtain*

$$\text{Adv}_{\text{HQC}_C.\text{PKE}_C, \text{PRF}, 0}^{\text{cUP}}(\mathcal{A}) \leq 2^{-\min(\ell, 8)}.$$

For our proposed HQC confirmation code, the unpredictability advantage above is essentially also the fCOR advantage upper bound, as the other terms in the bound of Theorem 16 are negligible in comparison.

Proof. Consider $\mathbf{v} = \text{truncate}(\mathbf{m}\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, \ell)$. The length of $\mathbf{s} \cdot \mathbf{r}_2$ is n bits, which is larger by ℓ bits than the length of $\mathbf{m}\mathbf{G}$. Thus, the truncation of $\mathbf{v}$ involves truncating ℓ bits of $\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$, where $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$, for secret values $\mathbf{x}, \mathbf{y} \in \mathcal{R}_w$. We show that the distribution of the last bits coefficients of $\mathbf{v}$ is uniform. Consider

$$\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e} = \mathbf{x} \cdot \mathbf{r}_2 + \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}_2 + \mathbf{e}.$$

Note that $\mathbf{h}$ is uniformly distributed, and $\mathbf{e}$ is uniformly distributed over strings of weight w_e . As was done in [2, 3], we make the simplifying assumption that the coordinates of $\mathbf{e}$ are independent, and

$\text{HQC}_C.\text{PKE}_C.\text{Enc}_C(\text{pk}, \mathbf{m} \in \mathbb{F}_2^{128}; r)$	$\text{HQC}_C.\text{Encaps}(\text{pk})$
01 $\mathbf{r}_1 \leftarrow \text{PRF}(r, w_r, n, 1)$	09 $\text{salt} \leftarrow \mathbb{F}_2^{128}$
02 $\mathbf{r}_2 \leftarrow \text{PRF}(r, w_r, n, 2)$	10 $\mathbf{m} \leftarrow \mathbb{F}_2^{128}$
03 $\mathbf{e} \leftarrow \text{PRF}(r, w_e, n, 3)$	11 $\theta \leftarrow \mathcal{G}(\mathbf{m} \parallel \text{firstBytes}(\text{pk}, 32) \parallel \text{salt})$
04 $\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$	12 $(\mathbf{c}, \mathbf{cd}) \leftarrow \text{HQC}_C.\text{PKE}_C.\text{Enc}_C(\text{pk}, \mathbf{m}; \theta)$
05 $\mathbf{v} \leftarrow \text{truncate}(\mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, \ell)$	13 $K \leftarrow \mathcal{G}(\mathbf{m}, \mathbf{c}, \mathbf{cd})$
06 $\mathbf{cd} \leftarrow \text{lastBytes}(\mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, 1)$	14 return $(K, c = \mathbf{c} \parallel \text{salt})$
07 $\mathbf{c} \leftarrow (\mathbf{u}, \mathbf{v})$	
08 return $(\mathbf{c}, \mathbf{cd})$	
	$\text{HQC}_C.\text{Decaps}(\text{sk}, c = \mathbf{c} \parallel \text{salt})$
	15 $\mathbf{m}' \leftarrow \text{HQC}.\text{Dec}(\text{sk}, \mathbf{c})$
	16 $\theta' \leftarrow \mathcal{G}(\mathbf{m}' \parallel \text{firstBytes}(\text{pk}, 32) \parallel \text{salt})$
	17 $(\mathbf{c}', \mathbf{cd}') \leftarrow \text{HQC}_C.\text{PKE}_C.\text{Enc}_C(\text{pk}, \mathbf{m}'; \theta')$
	18 $K \leftarrow \mathcal{G}(\mathbf{m}, \mathbf{c}, \mathbf{cd}')$
	19 $\bar{K} \leftarrow \mathcal{G}(z, \mathbf{c})$
	20 if $\mathbf{m}' = \perp$ or $\mathbf{c} \neq \mathbf{c}'$:
	21 return $\bar{K}$
	22 return K

Fig. 15: Confirmation-code-augmented version HQC_C of HQC , with the underlying confirmation-code-augmented encryption algorithm $\text{HQC}_C.\text{PKE}_C.\text{Enc}_C$ using function PRF . Changes introducing confirmation codes are highlighted in **violet boxes**.

thus $\Pr[\mathbf{e}_k = 1]$ follows a Bernoulli distribution with parameter $p = \frac{w}{n}$.⁸ Let $\bar{\mathbf{x}} = \mathbf{x} \cdot \mathbf{r}_2$ and $\bar{\mathbf{y}} = \mathbf{y} \cdot \mathbf{r}_2$. From Proposition 22 we have that each coordinate of $\bar{\mathbf{x}}$ and $\bar{\mathbf{y}}$ follows a Bernoulli distribution with the following parameter $\tilde{p}$ describing the distribution of coefficients:

$$\tilde{p} = \frac{1}{\binom{n}{w} \binom{n}{w_r}} \sum_{\substack{1 \leq \ell \leq \min(w, w_r) \\ \ell \text{ odd}}} \binom{n}{\ell} \binom{n-\ell}{w-\ell} \binom{n-w}{w_r-\ell}.$$

Now, let us consider the output distribution of $\mathbf{h} \cdot \bar{\mathbf{y}}$. We have

$$(\mathbf{h} \cdot \bar{\mathbf{y}})_k = \sum_{i+j \equiv k \pmod n} \mathbf{h}_i \bar{\mathbf{y}}_j.$$

Since $\mathbf{h}$ follows a uniform distribution, and $\bar{\mathbf{y}}$ follows a Bernoulli distribution with parameter $\tilde{p}$ we have that $\Pr[\mathbf{h}_i \bar{\mathbf{y}}_j = 1] = \frac{\tilde{p}}{2}$. Then the probability that $\sum_{i+j \equiv k \pmod n} \mathbf{h}_i \bar{\mathbf{y}}_j = 1$, is described by the probability that after $k+1$ independent Bernoulli trials, the number of successes is odd. A ‘success’ in this case occurs with probability $\frac{\tilde{p}}{2}$. We have

$$\Pr[(\mathbf{h} \cdot \bar{\mathbf{y}})_k = 1] = \frac{1}{2} - \frac{1}{2} (1 - \tilde{p})^{k+1}.$$

Now we consider the distribution of $\mathbf{z} = \mathbf{x} \cdot \mathbf{r}'_2 + \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}'_2 + \mathbf{e}'$. We consider $\Pr[\mathbf{z}_k = 1]$. Here, $\mathbf{z}_k$ is the sum modulo 2 of 3 terms, each distributed according to an independent Bernoulli distribution.

⁸ In reality, since $\mathbf{e}$ is of a fixed low weight, $\Pr[\mathbf{e}_k = 1]$ depends on the weight of $(\mathbf{e}_0, \dots, \mathbf{e}_{k-1})$.

Table 2: Performance impact of confirmation code on ML-KEM and HQC.
Measured with `liboqs v0.12.1-dev` using `mlkem-native` [27] `aarch64` on an Apple Silicon M2 Max processor.

Algorithm		Short conf. code		Full-length conf. code	
		Size (bytes)	Slowdown	Size (bytes)	Slowdown
ML-KEM-512	Encaps	12	3.4%	1152	23.6%
	Decaps		3.1%		23.6%
ML-KEM-768	Encaps	16	2.1%	1536	20.5%
	Decaps		2.1%		15.0%
ML-KEM-1024	Encaps	20	1.2%	1920	14.1%
	Decaps		1.6%		12.2%
HQC-128 with bug	Encaps	1	0.09%	—	—
	Decaps		0.25%		—
HQC-128 bug fixed	Encaps	1	0.13%	—	—
	Decaps		0.19%		—

Thus we have

$$\begin{aligned}
\Pr[\mathbf{z}_k = 1] &= \Pr[\mathbf{x} \cdot \mathbf{r}'_2 = 1, \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}'_2 = 1, \mathbf{e}'_k = 1] \\
&\quad + \Pr[\mathbf{x} \cdot \mathbf{r}'_2 = 1, \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}'_2 = 0, \mathbf{e}'_k = 0] \\
&\quad + \Pr[\mathbf{x} \cdot \mathbf{r}'_2 = 0, \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}'_2 = 1, \mathbf{e}'_k = 0] \\
&\quad + \Pr[\mathbf{x} \cdot \mathbf{r}'_2 = 0, \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}'_2 = 0, \mathbf{e}'_k = 1].
\end{aligned}$$

We observe computationally $\Pr[\mathbf{z}_k = 1]$ converges to 0.5. Concretely, we have

- For HQC-128, $n = 17,699$, $w = 66$, $w_e = w_r = 75$, $\tilde{p} = 0.2157612$, and $\ell = 5$ and $\Pr[\mathbf{z}_k = 1] = 0.5$ for $k > 140$.
- For HQC-192, $n = 35,851$, $w = 100$, $w_e = w_r = 114$, $\tilde{p} = 0.2362947$, and $\ell = 11$, and $\Pr[\mathbf{z}_k = 1] = 0.5$ for $k > 125$.
- For HQC-256, $n = 57,637$, $w = 131$, $w_e = w_r = 149$, $\tilde{p} = 0.2468457$, and $\ell = 37$, and $\Pr[\mathbf{z}_k = 1] = 0.5$ for $k > 119$.

We conclude that the last ℓ bits of $\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$ used as confirmation code are uniformly random, and independent of the view of the $\mathcal{A}$ and so

$$\text{Adv}_{\text{HQC}_C.\text{PKE}_C,\text{PRF},0}^{\text{cUP}}(\mathcal{A}) \leq \frac{1}{2^{\min(\ell,8)}}.$$

□

5.3 Performance Evaluation

ML-KEM. We adapted the ML-KEM implementation in `liboqs` [24, 29] (based on `mlkem-native` [27]) to include confirmation codes as specified in Figs. 13 and 14. We evaluated two options: a short confirmation code, using the first and last coefficient of uncompressed ciphertext vector for the confirmation code (i.e., $S = \{1, 256\}$), and a full-length confirmation code (i.e., $S = \{1, \dots, 256\}$). Our performance evaluation (see Table 2 for details) shows that using the short confirmation code leads to

a slow-down in the encapsulation and decapsulation algorithms of at most 3.4% across ML-KEM-512, ML-KEM-768, and ML-KEM-1024. Using the entire uncompressed ciphertext for the confirmation code has a much more significant performance impact due to the cost of hashing such a long input into key derivation, relative to the otherwise low cost of ML-KEM encapsulation/decapsulation. Fortunately, as we have shown in Theorem 21, the short confirmation code ($|S| = 2$) already suffices to obtain a cUP advantage upper bound that ensures faulty implementations will fail noticeably (with probability about $1/3$, see Table 1) in each test run.

HQC. We likewise augmented the HQC implementation in `liboqs` [24, 29] with confirmation codes as specified in Fig. 15 and evaluated it for the HQC-128 parameter set. Using the last byte of as confirmation code (for efficiency, given all the APIs are byte-oriented), we measured a slow-down in encapsulation and decapsulation of at most 0.25%. We applied confirmation codes to both the version of code *with* the re-encryption bug [25] and the patched version with the bug fixed, and experimentally confirmed that our approach works: the 1-byte confirmation code indeed detected the flaw in the buggy implementation, reliably causing the basic correctness tests of `liboqs` to fail.

6 Conclusion and Discussion

In this paper, we bring the methodology of cryptographically tying security to functional correctness through confirmation codes, proposed by Fischlin and Günther [9] in the form of “verifiable verification” for digital signatures, to the realm of key encapsulation mechanisms (KEMs). We developed a modified Fujisaki–Okamoto (FO) transform that binds re-encryption to functionality, making the correct implementation of the FO re-encryption verifiable for implementers. The core idea is to export an unpredictable confirmation code from the underlying PKE scheme and include it in the key derivation. We showed that this approach ensures that a faulty implementation which skips re-encryption will be noticeably incorrect, and hence exposed through basic correctness tests. We applied this technique to ML-KEM and HQC to present confirmation-code-augmented variants that leverage the entropy lost through ciphertext compression or truncation with negligible overhead, and experimentally confirmed that this approach successfully surfaces the re-encryption implementation flaw in HQC’s reference implementation.

PERMANENT SAFETY NET OR TEMPORARY BUG TESTING? One could see verifiable decapsulation as a property that is permanently integrated into KEM designs. In this way, one not only captures local implementation errors before deployment, but can also detect flaws in a remote KEM implementation when used, e.g., within a larger protocol. Also, any future code modifications would automatically benefit from the technique, providing a permanent safety net.

On the other hand, one could see verifiable decapsulation as a bug-testing technique. In this view, one would use confirmation codes in pre-deployment version of code for enhanced correctness tests. The deployed code version would then remove the confirmation codes again, reverting to the standard KEM scheme. This approach avoids the overhead introduced by confirmation codes. However, maintaining two implementations that differ in lower-level cryptographic instructions may itself be prone to errors, and skipping such a step could be a development “optimization” taken by a busy implementer.

We tend to view verifiable decapsulation as a design principle permanently embedded in KEMs, and see our experimental results on ML-KEM and HQC as supporting this, demonstrating that the overhead introduced is very small.

FUTURE WORK. In order to model how a faulty implementation might behave, we presented a notion of confirmation code unpredictability that gave the adversary access to all inputs and instead

limited its access to some internal function F . While this approach avoids trivial confirmation codes possible in [9] (like the message which is PKE-encrypted) and captures an incomplete re-encryption implementation step well in the KEMs we study, not every cryptographic scheme admits identifying such function F , limiting access to which would match intuition. Not all post-quantum KEMs we looked at admit natural confirmation codes in our paradigm. For example, FrodoKEM [22] does not use ciphertext compression or truncation like ML-KEM or HQC which we leveraged for obtain unpredictable confirmation codes. It would be interesting to explore different formalisms for restricting which computations a faulty implementation may perform, through limiting input access, computation time, etc. Likewise, exploring further applications of the paradigm of “tying security to basic functionality” to cover other forms of faults in decryption algorithms (beyond FO) would be interesting direction.

References

1. Aguilar-Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Bos, J., Deneuville, J.C., Dion, A., Gaborit, P., Lacan, J., Persichetti, E., Robert, J.M., Véron, P., Zémor, G.: Hamming Quasi-Cyclic (HQC), fourth round version, updated version 19/02/2025 (Feb 2025), https://pqc-hqc.org/doc/hqc-specification_2025-02-19.pdf
2. Aguilar-Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Persichetti, E., Zémor, G., Bos, J., Dion, A., Lacan, J., Robert, J.M., Veron, P.: HQC. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>
3. Aragon, N., Gaborit, P., Zémor, G.: HQC-RMRS, an instantiation of the HQC encryption framework with a more efficient auxiliary error-correcting code (2020), <https://arxiv.org/abs/2005.10741>
4. Bellare, M., Davis, H., Günther, F.: Separate your domains: NIST PQC KEMs, oracle cloning and read-only indistinguishability. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 3–32. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_1
5. Bernstein, D.J., Persichetti, E.: Towards KEM unification. Cryptology ePrint Archive, Report 2018/526 (2018), <https://eprint.iacr.org/2018/526>
6. Bindel, N., Hamburg, M., Hövelmanns, K., Hülsing, A., Persichetti, E.: Tighter proofs of CCA security in the quantum random oracle model. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019, Part II. LNCS, vol. 11892, pp. 61–90. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-36033-7_3
7. Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Stehlé, D.: CRYSTALS – Kyber: a CCA-secure module-lattice-based KEM. Cryptology ePrint Archive, Report 2017/634 (2017), <https://eprint.iacr.org/2017/634>
8. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Online-extractability in the quantum random-oracle model. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 677–706. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07082-2_24
9. Fischlin, M., Günther, F.: Verifiable verification in cryptographic protocols. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 3239–3253. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3623151>
10. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M.J. (ed.) CRYPTO’99. LNCS, vol. 1666, pp. 537–554. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_34
11. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. Journal of Cryptology **26**(1), 80–101 (Jan 2013). <https://doi.org/10.1007/s00145-011-9114-1>
12. Heninger, N.: Biased Nonce Sense: Lattice attacks against weak ECDSA signatures in the wild. Talk at the Workshop on Attacks in Cryptography 2 (WAC2), Crypto 2019 (Aug 2019), <https://crypto.iacr.org/2019/affevents/wac/medias/Heninger-BiasedNonceSense.pdf>
13. Heninger, N.: Implementation footguns for post-quantum cryptography. Talk at the Real World Post Quantum Workshop (RWPQC 2024) (Mar 2024), <https://na-admin.eventscloud.com/docs/9769/414552>

14. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017, Part I. LNCS, vol. 10677, pp. 341–371. Springer, Cham (Nov 2017). https://doi.org/10.1007/978-3-319-70500-2_12
15. Hövelmanns, K.: Generic constructions of quantum-resistant cryptosystems. doctoralthesis, Ruhr-Universität Bochum, Universitätsbibliothek (2021). <https://doi.org/10.13154/294-7758>
16. Hövelmanns, K., Hülsing, A., Majenz, C.: Failing gracefully: Decryption failures and the Fujisaki-Okamoto transform. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part IV. LNCS, vol. 13794, pp. 414–443. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22972-5_15
17. Hövelmanns, K., Kiltz, E., Schäge, S., Unruh, D.: Generic authenticated key exchange in the quantum random oracle model. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part II. LNCS, vol. 12111, pp. 389–422. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45388-6_14
18. Hövelmanns, K., Majenz, C.: A note on failing gracefully: Completing the picture for explicitly rejecting fujisaki-okamoto transforms using worst-case correctness. In: Saarinen, M.J., Smith-Tone, D. (eds.) Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II. pp. 245–265. Springer, Cham (Jun 2024). https://doi.org/10.1007/978-3-031-62746-0_11
19. Jiang, H., Zhang, Z., Chen, L., Wang, H., Ma, Z.: IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part III. LNCS, vol. 10993, pp. 96–125. Springer, Cham (Aug 2018). https://doi.org/10.1007/978-3-319-96878-0_4
20. Jiang, H., Zhang, Z., Ma, Z.: Key encapsulation mechanism with explicit rejection in the quantum random oracle model. In: Lin, D., Sako, K. (eds.) PKC 2019, Part II. LNCS, vol. 11443, pp. 618–645. Springer, Cham (Apr 2019). https://doi.org/10.1007/978-3-030-17259-6_21
21. Kuchta, V., Sakzad, A., Stehlé, D., Steinfeld, R., Sun, S.: Measure-rewind-measure: Tighter quantum random oracle model proofs for one-way to hiding and CCA security. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part III. LNCS, vol. 12107, pp. 703–728. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45727-3_24
22. Naehrig, M., Alkim, E., Bos, J., Ducas, L., Easterbrook, K., LaMacchia, B., Longa, P., Mironov, I., Nikolaenko, V., Peikert, C., Raghunathan, A., Stebila, D.: FrodoKEM. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
23. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. Federal Information Processing Standards Publications (FIPS PUBS) 203, U.S. Department of Commerce, Washington, D.C. (2024). <https://doi.org/10.6028/NIST.FIPS.203>
24. Open Quantum Safe project: liboqs, <https://openquantumsafe.org/liboqs/>
25. Open Quantum Safe project: Correctness error in HQC decapsulation. Reported by Célian Glénaz and Dahmun Goudarzi. CVE-2024-54137 (Dec 2024), <https://github.com/open-quantum-safe/liboqs/security/advisories/GHSA-gpf4-vrrw-r8v7>
26. Poulsen, K.: Behind iPhone’s critical security bug, a single bad ‘goto’. <https://www.wired.com/2014/02/gotofail/> (Feb 2014)
27. PQ Code Package project: mlkem-native, <https://github.com/pq-code-package/mlkem-native/>
28. Saito, T., Xagawa, K., Yamakawa, T.: Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part III. LNCS, vol. 10822, pp. 520–551. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78372-7_17
29. Stebila, D., Mosca, M.: Post-quantum key exchange for the internet and the open quantum safe project. In: Avanzi, R., Heys, H.M. (eds.) SAC 2016. LNCS, vol. 10532, pp. 14–37. Springer, Cham (Aug 2016). https://doi.org/10.1007/978-3-319-69453-5_2

A Recalling the OW-CPA bounds for the T-transform given in [14]

We now restate [14, Thms. 3.1 and 3.2]. In the theorems, PKE_1 denotes the PKE scheme $\text{T}[\text{PKE}, \text{G}]$, and OW-CPA-PCVA denotes OW-CPA security even in the presence of two additional oracles PCO and CVO . Simplifying the given bounds as indicated in Theorem 13 by dropping the additional oracles is straightforward, but we still detail this below the theorems for the sake of completeness.

Theorem 24 ([14, Theorem 3.1]). *If PKE is δ -correct, then PKE_1 is δ_1 -correct in the random oracle model with $\delta_1(q_G) = q_G \cdot \delta$. Assume PKE to be γ -spread. Then, for any OW-CPA-PCVA adversary $\mathcal{B}$ that issues at most q_G queries to the random oracle $\mathcal{G}$, q_P queries to a plaintext checking oracle PCO, and q_V queries to a validity checking oracle CVO, there exists an OW-CPA adversary $\mathcal{A}$ such that*

$$\text{Adv}_{\text{PKE}_1}^{\text{OW-CPA-PCVA}}(\mathcal{B}) \leq (q_G + q_P) \cdot \delta + q_V \cdot 2^{-\gamma} + (q_G + q_P + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A})$$

and the running time of $\mathcal{A}$ is about that of $\mathcal{B}$. Furthermore, PKE_1 is rigid.

Theorem 25 ([14, Theorem 3.2]). *Assume PKE to be δ -correct and γ -spread. Then, for any OW-CPA-PCVA adversary $\mathcal{B}$ that issues at most q_G queries to the random oracle $\mathcal{G}$, q_P queries to a plaintext checking oracle PCO, and q_V queries to a validity checking oracle CVO, there exists an IND-CPA adversary $\mathcal{A}$ such that*

$$\text{Adv}_{\text{PKE}_1}^{\text{OW-CPA-PCVA}}(\mathcal{B}) \leq (q_G + q_P) \cdot \delta + q_V \cdot 2^{-\gamma} + \frac{2q_G + 1}{|\mathcal{M}|} + 3 \cdot \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A})$$

and the running time of $\mathcal{A}$ is about that of $\mathcal{B}$.

In Theorem 13, we are only interested in plain OW-CPA security. By definition, OW-CPA security is OW-CPA-PCVA security with $q_P = 0$ queries to the additionally provided plaintext checking oracle PCO and $q_V = 0$ queries to the validity checking oracle CVO, thus the term $(q_G + q_P + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A})$ in Theorem 3.1 simplifies to $(q_G + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A})$.

The two terms involving δ and γ are shared by both bounds and account for how both proofs simulate the two additional oracles PCO and CVO. Since we are only interested in plain OW-CPA security, we can dismiss the two oracles and the respective terms vanish.

B Proof that the TC-transform maintains code unpredictability (Proofs of Thm. 13)

We briefly repeat the proof's main idea: to an cUP adversary trying to predict the code belonging to a ciphertext, there is not too much of a difference between attacking PKE_C and its derandomized version $\text{TC}[\text{PKE}_C, \mathcal{G}]$. Since the only additional attack surface is that it might be more easy to invert derandomized ciphertexts, thus breaking OW-CPA security of $\text{T}[\text{PKE}, \mathcal{G}]$, we bound the success via a OW-CPA reduction both in the ROM (Theorem 13).

B.1 Proof of ROM Theorem 13

Consider the two games given in Fig. 16.

GAMES $G_0 - G_1$		$\mathcal{B}^G(\text{pk}, c^*)$
01 $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}$		08 $\mathcal{L}_G \leftarrow \emptyset$
02 $m \leftarrow_s \mathcal{M}$		09 $\text{cd}' \leftarrow \mathcal{A}_{\text{TC}[\text{PKE}_C^{\text{PRF}_G], \text{max}}}(\text{pk}, \text{sk}, m, c, r)$
03 $r \leftarrow \mathcal{G}(m)$	// G_0	10 $m' \leftarrow_s \mathcal{M}$
04 $r \leftarrow_s \mathcal{R}$	// G_1	11 for $m \in \mathcal{L}_G$
05 $(\text{cd}, c) \leftarrow \text{Enc}_C^{\text{PRF}}(\text{pk}, m; r)$		12 if $\text{Enc}(\text{pk}, m; \mathcal{G}(m)) = c^*$
06 $\text{cd}' \leftarrow \mathcal{A}^G(\text{pk}, c)$		13 $m' \leftarrow m$
07 return $\llbracket \text{cd} = \text{cd}' \rrbracket$		14 return m'

Fig. 16: Games G_0 and G_1 and adversary $\mathcal{B}$ for the proof of Theorem 13. List $\mathcal{L}_G$ stores $\mathcal{A}$'s random oracle queries to $\mathcal{G}$.

Game G_0 is the confirmation code unpredictability game against $\text{TC}[\text{PKE}_C^{\text{PRF}}G]$.

$$|\Pr[G_0 \Rightarrow 1]| = \text{Adv}_{\text{TC}[\text{PKE}_C, G]}^{\text{UP}}(\mathcal{A}) .$$

Game G_1 differs from G_0 by decoupling the involved randomness r from the message: when computing $(c, \text{cd}) \leftarrow \text{Enc}_C(\text{pk}, m; r)$, we now use a uniformly random value r that is independent of m instead of using $r \leftarrow G(m_0)$. In other words, we switch from the Enc_C algorithm of $\text{TC}[\text{PKE}_C, G]$ to that of PKE_C . Game G_1 thus is exactly the unpredictability game for the underlying augmented scheme PKE_C .

$$|\Pr[G_1 \Rightarrow 1]| = \text{Adv}_{\text{PKE}_C}^{\text{UP}}(\mathcal{A}) .$$

Since we model G as a random oracle, the change is unnoticed by $\mathcal{A}$ unless it queries G on m , thereby breaking one-way security of $\text{TC}[\text{PKE}, G]$. To formalize this, we start by calling the event **QUERY**.

$$|\Pr[G_1 \Rightarrow 1] - \Pr[G_0 \Rightarrow 1]| \leq \Pr[\text{QUERY}] .$$

We now define a **OW-CPA** adversary $\mathcal{B}$ against $\text{T}[\text{PKE}, G]$ in Fig. 16 that wins if **QUERY** occurs: $\mathcal{B}$ forwards its challenge ciphertext to $\mathcal{A}$. It forwards $\mathcal{A}$'s random oracle queries to G to its own random oracle, and also keeps track of these queries by storing them in a list $\mathcal{L}_G$. If **QUERY** occurs, the challenge plaintext appears in $\mathcal{L}_G$. Since $\text{T}[\text{PKE}, G]$ encrypts deterministically, $\mathcal{B}$ can go through the list after $\mathcal{A}$ has finished and identify the right plaintext.

$$\Pr[\text{QUERY}] \leq \text{Adv}_{\text{T}[\text{PKE}, G]}^{\text{OW-CPA}}(\mathcal{B}) .$$

Adding the inequalities gives the desired bound.

C IND-CCA security of our transformations UC_m^χ and FOC_m^χ

C.1 UC_m^χ is IND-CCA secure in the ROM

Our ROM theorem below establishes a bound that resembles the bound U_m^χ that was given in [15, Thm. 2.1.7], except that Thm.2.1.7 bounded correctness errors slightly differently.

Theorem 26 (ROM security of UC_m^χ). *Let PKE_C be a deterministic confirmation augmented PKE with code and ciphertext generation independent of G , and assume the underlying PKE to be δ -worst-case correct. If there are any random oracles used in the construction of PKE_C , let $q_{\text{Enc}, O}$ denote an upper bound on the number of random oracle queries that Enc_C trigger upon a single invocation. Let $H : \mathcal{PK} \rightarrow \{0, 1\}^\gamma$ be a fixed-output length function.*

Let $\mathcal{A}$ be an adversary against IND-CCA security of $\text{KEM}_C = \text{UC}_m^\chi[\text{PKE}_C, G, H, J]$, issuing at most q_D many queries to the decapsulation oracle for KEM_C , and at most q_O, q_G, q_J queries to the random oracle used in PKE and the random oracles G, J , respectively.

Then in the ROM, there exists a OW-CPA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_C}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{B}) + (q_O + q_G(q_{\text{Enc}, O}) + 1) \cdot \delta + \frac{q_J}{|\mathcal{M}|} ,$$

$\mathcal{B}$ issues at most $q_O + q_G \cdot (q_{\text{Enc}, O})$ many queries to oracle O , and the running time of $\mathcal{B}$ is about that of $\mathcal{A}$.

Proof. Our proof is close to the proof for $UC_m^\mathcal{J}$ that was given in [15]. Recall that we denote by $\text{Enc}(\text{pk}, m)$ the algorithm that runs $\text{Enc}_C(\text{pk}, m)$ and returns only the ciphertext. Likewise, let Code denote the algorithm that runs Enc_C and outputs only cd . We start by recalling its high-level idea and discussing how to adapt it to $UC_m^\mathcal{J}$: IND-CCA attacker $\mathcal{A}$ is given a challenge plaintext $c^* = \text{Enc}(\text{pk}, m^*)$ and a challenge key, which is either random or $G(m^*)$. In the ROM, $G(m^*)$ is indistinguishable from random unless $\mathcal{A}$ queries G on m^* , which can be leveraged by one-way reduction $\mathcal{B}$ that uses $\mathcal{A}$ to invert its challenge plaintext c^* . To help $\mathcal{B}$ simulate the oracle for Decaps without sk , the proof utilizes that Enc is deterministic – $\mathcal{B}$ uses Enc to recognize when a random oracle query m to G is connected to a decapsulation query c , in which case it outputs the same key. The simulation thus switches the connection between the oracles from $\text{Decaps}(\text{sk}, c) = G(\text{Dec}(\text{sk}, c))$ to $G(m) = \text{Decaps}(\text{sk}, \text{Enc}(\text{pk}, m))$. $\mathcal{A}$ only notices this when finding a message m for which $\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) \neq m$, which explains the δ -term.

To adapt the proof to $UC_m^\perp$, we note that we can easily integrate random oracle input $H(\text{pk})$ into the reasoning above since pk is a public value. To integrate cd , we leverage that Enc_C is assumed to be deterministic and one can thus easily identify the code that belongs to a plaintext. To enable the simulation of Decaps for $UC_m^\perp$, our proof essentially redefines the two oracles such that their connection switches from $\text{Decaps}(\text{sk}, c) = G(m', H(\text{pk}), \text{Enc}(\text{pk}, m'))$, where $m' = \text{Dec}(\text{sk}, c)$, to $G(m, H(\text{pk}), \text{Enc}(\text{pk}, m)) = \text{Decaps}(\text{sk}, \text{Enc}(\text{pk}, m))$.

We now recall the ROM proof of $U_m^\perp$ in more detail and discuss in the relevant places how we adapt it to accommodate the additional hash inputs.

Game 1. To prepare the simulation of the oracle for Decaps , the proof first replaces ‘implicit rejection’ keys with uniformly random values. This goes unnoticed unless $\mathcal{A}$ queries J on the rejection seed z , which happens with probability $q_J/|\mathcal{M}|$. This game needs no adapting.

Game 2. To prepare the simulation of the oracle for Decaps , the proof now ‘patches encryption into’ the lazily sampled random oracle G : upon a query m to G , the game computes the encryption c of m and stores c together with the randomly sampled key in an additional book-keeping list $\mathcal{L}_D$. Upon a query c to the decapsulation oracle, the game consults list $\mathcal{L}_D$ to keep $\mathcal{A}$ ’s view consistent, i.e., to return the key that was already associated to that ciphertext (if it already exists). Effectively, this switches the association from $\text{Decaps}(\text{sk}, c) = G(\text{Dec}(\text{sk}, c))$ to $G(m) = \text{Decaps}(\text{sk}, \text{Enc}(\text{pk}, m))$. Thus, the switch of association could only create an inconsistency in one of two cases:

- $\mathcal{A}$ queries G on a message that exhibits decryption failure – in that case, the proper input to G , m' , would differ from the chosen input m . This is reflected in the δ -term.
- $\mathcal{A}$ could request two distinct ciphertexts $c_1 \neq c_2$ that both decrypt to the same message m . In game 1, the decapsulation oracle would respond to both queries with the same value $G(m)$. This would not necessarily be the case in game 2. This inconsistency is prevented by requiring PKE to be rigid, i.e., by requiring that for any key pair and any ciphertext c , it always holds that $m' = \text{Dec}(\text{sk}, c) = \perp$ or $\text{Enc}(\text{pk}, m') = c$.

We now **adapt Game 2** to accommodate the additional hashing of confirmation code and public-key identifier $H(\text{pk})$. Intuitively, this works since pk is a public value and since we assume code generation to be deterministic: upon a query (m, h, cd) to G , the game checks if $h = H(\text{pk})$. If yes, it computes the code $\text{cd}' \leftarrow \text{Code}(\text{pk}, m)$ and checks if $\text{cd} = \text{cd}'$. If yes, it computes the encryption c of m and stores c together with the randomly sampled key in book-keeping list $\mathcal{L}_D$. Upon a decapsulation query, the game then again consults list $\mathcal{L}_D$ to keep $\mathcal{A}$ ’s view consistent. This switches the association from $\text{Decaps}(\text{sk}, c) = G(\text{Dec}(\text{sk}, c), H(\text{pk}), \text{Code}(\text{Dec}(\text{sk}, c)))$ to $G(m, H(\text{pk}), \text{Code}(\text{pk}, m)) = \text{Decaps}(\text{sk}, \text{Enc}(\text{pk}, m))$. With the same reasoning as in the original proof, we note that this goes unnoticed unless $\mathcal{A}$ queries G on a message that exhibits decryption failure, which is upper-bounded by the same δ -term as before. (We note that if no decryption failure occurred, then the confirmation codes $\text{Code}(\text{pk}, m)$ and $\text{Code}(\text{pk}, m')$ for $m' \leftarrow \text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m))$ will not differ for any queried message, thus not leading to any inconsistency.)

Note that we do not require PKE to be rigid because unlike U_m^χ , our transformation UC_m^χ performs re-encryption – if $\mathcal{A}$ requests two distinct ciphertexts $c_1 \neq c_2$ that both decrypt to the same message m , at least one of them will land in the implicit rejection branch, meaning their decapsulations would already have differed in game 1.

Game 3. The last game argues indistinguishability of the challenge session key via one-wayness of PKE: Game 3 raises flag CHAL and immediately aborts on the event that $\mathcal{A}$ queries G on the input that would generate the honest challenge key K_0 , so on the input m^* with m^* being the challenge plaintext. (In our case, this changes to the input $(m^*, H(pk), \text{Code}(pk, m^*))$ with m^* being the challenge plaintext.) If this query never occurs, $\mathcal{A}$ never sees the honest challenge key, thus has no chance distinguishing it from random beyond random guessing, $\mathcal{A}$'s success probability in game 3 thus is $1/2$. The change in $\mathcal{A}$'s success probability between games 2 and 3 is upper-bounded by the probability of CHAL, and event CHAL can be used to break the one-wayness of PKE. Our adapted adversary $\mathcal{B}$ simulates game 3 for $\mathcal{A}$. Since Enc and code generation are assumed to be deterministic, $\mathcal{B}$ is able to recognize CHAL and thus m^* : upon each of $\mathcal{A}$'s G queries $(m, H(pk), cd)$, $\mathcal{B}$ checks if $\text{Code}(pk, m) = cd$ and, if yes, if $\text{Enc}(pk, m)$ equals its challenge ciphertext c^* . If yes, it immediately aborts $\mathcal{A}$ and returns m to its OW-CPA game. Since Enc is assumed to be deterministic, we have $\text{Enc}(pk, m) = c$ and $\mathcal{B}$ thus returns m^* unless m^* exhibits decryption failure, which happens with probability at most δ . $\square$

C.2 UC_m^χ is also IND-CCA secure in the QROM

Our second theorem adapts the ROM result, Theorem 26, to the QROM. It essentially recovers the QROM security bound for U^χ that was given in [6, Thm. 2] for our augmented variant.

Theorem 27 (QROM security of UC_m^χ). *Let G be a quantum-accessible random oracle, let $H : \mathcal{PK} \rightarrow \{0, 1\}^\gamma$ be a fixed-output length function, and let $J : \mathcal{K}_J \times \mathcal{C} \rightarrow \mathcal{K}$ be a PRF. Let PKE_C be a deterministic confirmation augmented PKE with code and ciphertext generation both are independent of G , and let PKE denote the PKE scheme associated with PKE_C (i.e. it uses as Enc the algorithm that runs Enc_C and only outputs c). Assume PKE to be ϵ -injective, i.e., assume that*

$$\Pr[\text{Enc}(pk, -) \text{ is not injective}] \leq \epsilon ,$$

where the probability is being taken over the internal coins of KeyGen and the choice of the random oracles involved in the construction of PKE_C (if any).

Let $\mathcal{A}$ be an IND-CCA adversary against $\text{KEM}_C = UC_m^\chi[\text{PKE}_C, G, H, J]$ that issues at most q_D decapsulation queries. Then there exist three adversaries with about the same time/resources as $\mathcal{A}$:

- an OW-CPA adversary B_1 against PKE;
- an FFC adversary B_2 against PKE, returning a list of at most q_D many ciphertexts; and
- a PRF adversary B_3 against J

such that

$$\text{Adv}_{\text{KEM}_C}^{\text{IND-CCA}}(\mathcal{A}) \leq 2\sqrt{\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(B_1) + \text{Adv}_{\text{PKE}}^{\text{FFC}}(B_2) + 2\text{Adv}_J^{\text{PRF}}(B_3) + \epsilon} .$$

Remark 28. Before discussing the proof, we note that for perfectly correct schemes, $\text{Enc}(pk, -)$ will be non-injective with probability 0; the ϵ -term thus vanishes in this case. We also note that for PKE schemes that are deterministic, perfectly correct and that satisfy a stronger security property called *disjoint simulatability*, it may be possible to achieve a tighter QROM security bound by re-doing/adapting the proof of [28, Thm. 4.2].

Proof. The proof closely follows a proof [6, Thm. 2] for transformation U^χ . We will thus first recap that proof and then discuss how to adapt it.

On a high level, the proof for U^χ resembles the ROM reasoning above: base indistinguishability of the challenge session key on OW-CPA security, and to do so, replace $\mathcal{A}$'s oracles with sk-independent simulations. For the transformation U^χ , this means redefining the oracles such that $G(m, c) = \text{Decaps}(\text{sk}, \text{Enc}(\text{pk}, m))$ instead of $\text{Decaps}(\text{sk}, c) = G(\text{Dec}(\text{sk}, c), c)$. Since the random oracle queries now are in superposition, the proof needs to adapt how to

1. simulate G and Decaps without sk in a consistent way (no bookkeeping);
2. reason about the cases in which the simulation fails; and
3. find the challenge message m^* within the random oracle queries.

To deal with #1, the proof for U^χ ‘patches encryption into’ the random oracle by redefining $G(m, c) = R(c)$ whenever $c = \text{Enc}(\text{pk}, m)$, where R is a random function. With this change, the proof can simulate Decaps by setting $\text{Decaps}(\text{sk}, c) = R(c)$.

The proof for U^χ notes that the simulation fails (#2) in three cases: a) the redefined G no longer produces independent responses for distinct messages, which is prevented by requiring that $\text{Enc}(\text{pk}, -)$ is ϵ -injective. b) $\mathcal{A}$ requests a c that goes into the rejection branch and is able to distinguish $R(c)$ from $J(z, c)$, which is captured via PRF. c) $\mathcal{A}$ requests a decapsulation of a ciphertext that doesn't decrypt to its originating plaintext, which is captured via FFC.

To deal with #3, the proof for U^χ utilizes double-sided OwtH (Lemma 8), which incurs a quadratic loss. In the context of this proof, Lemma 8 states that any attacker $\mathcal{A}$ who can distinguish the session key $G(m^*, c^*)$ from uniform can be turned into an algorithm $\mathcal{B}$ that outputs m^* .

We now adapt this proof to our transformation UC_m^χ by making the following changes: We simulate G via $G(m, h, \text{cd}) \leftarrow R(\text{Enc}(\text{pk}, m))$ whenever $h = H(\text{pk})$ and $\text{cd} = \text{Enc}(\text{pk}, m)$. This simulation fails in exactly the same cases as in the non-adapted proof and thus leads to the same analysis. After switching to the simulation, we can again apply double-sided One-way-to-Hiding (with our adapted function G) to find the challenge message m^* within $\mathcal{A}$'s random oracle queries. $\square$

C.3 FOC_m^χ is IND-CCA secure in the ROM

Our third theorem, Corollary 29 below, essentially recovers the ROM security bound for FOC_m^χ that was given in [15, Sect. 2.1.4] for our augmented variant. Since $\text{FOC}_m^\chi = UC_m^\chi \circ \text{TC}$, Corollary 29 is obtained by combining Theorem 26 with previous results about T that were given in [14].

Corollary 29 (ROM security of FOC_m^χ). *Let PKE_C be a probabilistic confirmation augmented PKE scheme, and assume PKE to be δ -worst-case correct. Let $H : \mathcal{PK} \rightarrow \{0, 1\}^\gamma$ be a fixed-output length function. Let $\text{KEM}_C = \text{FOC}_m^\chi[\text{PKE}_C, G, H, J, G']$.*

Let $\mathcal{A}$ be an adversary against IND-CCA security of KEM_C , issuing at most q_D many queries to the decapsulation oracle for KEM_C , and at most $q_G/q_{G'}/q_J$ queries to the random oracles $G/G'/J$.

According to Theorem 26, in the ROM there exists a OW-CPA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_C}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{T[\text{PKE}, G']}^{\text{OW-CPA}}(\mathcal{B}) + (q_{G'} + q_G + 1) \cdot \delta + \frac{q_J}{|\mathcal{M}|} ,$$

$\mathcal{B}$ issues at most $q_{G'} + q_G$ many queries to oracle G' . According to [14], there thus exists a OW-CPA adversary $\mathcal{C}_O$ and an IND-CPA adversary $\mathcal{C}_I$ against $T[\text{PKE}, G']$ such that

$$\text{Adv}_{T[\text{PKE}, G']}^{\text{OW-CPA}}(\mathcal{B}) \leq \begin{cases} (q_{G'} + q_G + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{C}_O) \\ 3 \cdot \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{C}_I) + \frac{2q_{G'} + q_G + 1}{|\mathcal{M}|} \end{cases} ,$$

The running time of $\mathcal{B}$, $\mathcal{C}_O$ and $\mathcal{C}_I$ is about that of $\mathcal{A}$.

C.4 FOC_m^χ is IND-CCA secure in the QROM

Theorem 30 below essentially recovers the QROM security bound for FO_m^χ that was implicitly given in [16] for our augmented variant.

Theorem 30. *Let PKE_C be a (randomized) augmented PKE scheme that is γ -spread, let $H : \mathcal{PK} \rightarrow \{0,1\}^\gamma$ be a fixed-output length function, and let $\text{KEM}_C = \text{FOC}_m^\chi[\text{PKE}_C, G, H, J, G']$, and let PKE denote the PKE scheme associated to PKE_C . Let $\mathcal{A}$ be an IND-CCA adversary against KEM_C , making at most q_D many queries to its decapsulation oracle, and making $q_{G'}$, q_G queries to its respective random oracles. Furthermore, let $q = q_{G'} + q_G$, and let d and w be the query depth and query width of the combined queries to G' and G .*

Then there exist an IND-CPA adversary $\mathcal{B}_{\text{IND}}$, a OW-CPA adversary $\mathcal{B}_{\text{OW-CPA}}$ against PKE and an FFP-CPA adversary $\mathcal{C}$ against $T[\text{PKE}, G']$ in the extractable QROM with extractor function Enc such that

$$\text{Adv}_{\text{KEM}_C}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}} + (q_D + 1) \cdot \text{Adv}_{\text{PKE}}^{\text{FFP-CCA}}(\mathcal{C}) + \varepsilon_\gamma, \text{ with} \quad (2)$$

$$\text{Adv}_{\text{PKE}} = \begin{cases} 4 \cdot \sqrt{(d + q_D) \cdot \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{B}_{\text{IND}}) + \frac{8(q + q_D)}{\sqrt{|\mathcal{M}|}}} \\ 8(d + q_D) \cdot \sqrt{w \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{B}_{\text{OW-CPA}})}. \end{cases} \quad (3)$$

The additive error term ε_γ is given by

$$\varepsilon_\gamma = 24q_D(q_G + 4q_D)2^{-\gamma/2}.$$

Remark 31. Before discussing the proof, we note that [16] show how to bound $\text{Adv}_{\text{PKE}}^{\text{FFP-CCA}}(\mathcal{C})$ with a fine-grained analysis. Alternatively, [18] gives a simpler, albeit heuristic-prone bound by bounding

$$\text{Adv}_{\text{PKE}}^{\text{FFP-CCA}}(\mathcal{C}) \leq 10(q + q_D + 1)^2 \cdot \delta,$$

where δ is the correctness term of PKE as defined in Definition 1.

Proof. To summarize the proof for FO_m^χ in [16], the proof proceeds in two steps with their own separate theorems:

1. The first step mutes the decapsulation oracle, i.e., it bounds the difference between IND-CCA and IND-CPA security of FO_m^χ .
2. The second step bases IND-CPA security of FO_m^χ on passive security of PKE.

For the first step, [16] showed how to simulate the decapsulation oracle without the secret key. To that end, the randomness-generating random oracle G' is modeled as an extractable QRO. The extractable QRO provides not only the random oracle interface, but additionally an ‘extraction’ interface that (essentially) finds the right message for any queried ciphertext: the extraction interface can be queried on any ciphertext c and returns either $\perp$ or a originating message, i.e., a message m such that $\text{Enc}(\text{pk}, m; G'(m)) = c$. Using the extraction interface, the simulated decapsulation oracle can (essentially) decrypt the ciphertext without the secret key, and then derive the corresponding session key like the original decapsulation oracle. This simulation fails in two cases: a) the attacker submits a ciphertext such that the extraction interface does not find its preimage, which is captured via γ -spreadness. b) the attacker submits a ciphertext for which the originating message and the decrypted message do not match, which is captured via the ‘find failing plaintext’ property FFP-CCA.

The only differences in our setting are: instead of having an encryption algorithm Enc , we have a code-augmented encryption algorithm Enc_C , and when deriving the key, we additionally hash the

obtained confirmation code and the public-key identifier $H(pk)$. We can easily integrate random oracle input $H(pk)$ into that reasoning since pk is a public value. To integrate cd , we will use the same extraction interface as the proof for $FO_m^\perp$. (We can define the ‘pure encryption’ algorithm Enc of our augmented scheme PKE_C as the algorithm that runs Enc_C and drops the confirmation code.) With that, our simulation of the decapsulation oracle can again obtain the originating plaintext m of any queried ciphertext c . The simulation can then also acquire the associated confirmation code since $FOC_m^\perp$ derandomizes the algorithm Enc_C : the simulation can simply compute $(c', cd) := Enc_C(pk, m; G'(m))$. (One might think that this costs one more call to G' , but this call is anyways being done to be able to perform the re-encryption check.)

The second step simply argues that the resulting key is indistinguishable from random due to One-Way To Hiding (in the extractable QROM). Since we did not change the extractor function, this part can remain unchanged, except that we append $H(pk)$ and cd to the inputs of the random oracle G . $\square$